SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

Code of Conduct:

I R.SHOBANA SHRI(1921260053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:R.Shobana shri

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3

3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5

9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Error Analysis Task

1.A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

Auditing a Flawed Equivalence Partitioning and BVA Design for an Online Shop

Assumption: no test design was attached, so I built a representative flawed one. Send yours and I'll re-audit it line by line. The error taxonomy and the corrections apply to any input-range design.

Spec assumed (the oracle):

Field	Rule
Quantity	Integer 1 to 10 per item
Coupon discount	0% to 50% (whole numbers)
Age (alcohol category)	18 to 120
Order total for free shipping	≥ ₹999
Card PIN/CVV	exactly 3 digits
Pincode	exactly 6 digits

1. The Flawed Test Design (as given)

Field	Tester's "classes"	Test values chosen
Quantity	**Class A:** 0 to 10 · **Class B:** above 10	5, 15
Coupon %	**Class A:** -10 to 50 · **Class B:** 51 to 100	25, 75
Age	**Class A:** 1 to 120	30
Free shipping	**Class A:** 500 to 2000	1,200
CVV	**Class A:** valid (3 digits) · **Class B:** invalid	123, 12
Pincode	**Class A:** any numeric	600001

No boundary values were selected, and each class got exactly one test.

2. Error Inventory

2.1 Equivalence partitioning errors

#	Field	Error	Why it is wrong	Type
E1	Quantity	Class A "0 to 10" mixes 0 (invalid) with 1 to 10 (valid)	Members of one class must behave identically, but 0 is rejected and 5 is accepted	**Mixed valid/invalid**
E2	Quantity	No class for negatives, decimals, non-numeric, blank	Entire invalid families are missing	**Missing invalid classes**
E3	Coupon	Class A "-10 to 50" mixes negative (invalid) with 0 to 50 (valid)	A -10% coupon would raise the price, so it behaves differently	**Mixed valid/invalid**
E4	Coupon	Class B "51 to 100" is one invalid class, but 100 may behave as a special "free order" case	Possible sub-partition hidden	**Over-merged class**

E5	Age	Class A "1 to 120" labels ages 1 to 17 as valid	Minors must be blocked, so a single class hides the main rule	**Wrong validity label**
E6	Age	No class for above 120, zero, negative	Missing invalid classes	**Missing classes**
E7	Free shipping	Range 500 to 2000 straddles the ₹999 threshold	Values below and above 999 give different shipping results	**Class spans a decision boundary**
E8	Free shipping	No class for zero or empty cart, or above a maximum	Missing special cases	**Missing classes**
E9	CVV	Class B "invalid" is one lump	Too short, too long, letters and blank are different failures	**Under-partitioned invalid class**
E10	Pincode	"Any numeric" ignores length	5-digit and 7-digit numerics are invalid	**Class defined by wrong attribute**
E11	All	Exactly one test per class	Fine in theory for a correct class, useless when the class is wrong	**Compounds E1 to E10**

2.2 Boundary value analysis errors

#	Error	Detail
B1	**No boundaries at all**	Chosen values (5, 25, 30, 1,200, 123) are mid-range
B2	Quantity boundaries absent	0, 1, 10, 11 never tried
B3	Coupon boundaries absent	-1, 0, 50, 51 never tried
B4	Age boundaries absent	17, 18, 120, 121 never tried
B5	Threshold boundary absent	998, 999, 1000 never tried
B6	Length boundaries absent	CVV and pincode 2/3/4 and 5/6/7 digits never tried
B7	**Invalid-side values chosen far from the edge**	15, 75 and 12 prove little. An off-by-one fault (<= for <) survives
B8	No type or format boundaries	Leading zeros ("007"), "1e1", " 5", full-width digits

2.3 Defects the flawed design would let through

Seeded defect	Caught?	Why not
Quantity check qty >= 0 instead of >= 1	No	0 was never tested, and class A "0 to 10" blesses it
Quantity check qty < 10 instead of <= 10	No	10 never tried
Coupon accepts -5%	No	-10 sits in the "valid" class, so it is expected to pass
Age check age > 18 instead of >= 18	No	18 never tried
Age 17 allowed	No	Class A labels 1 to 17 valid
Free shipping at > 999 instead of >= 999	No	999 never tried
CVV accepts 4 digits	No	Only 2 digits tested on the invalid side
Pincode accepts 5 digits	No	Only a valid 6-digit value was used

All eight escape, so the design's expected defect detection is close to zero for edge faults.

3. Root-Cause Analysis

Root cause	Mistakes it explains	Reasoning
RC1: Partitioning by the data type, not by behavior	E1, E3, E10	The tester thought "numbers", but a class must group inputs the system treats the same. Type is not behavior
RC2: Partitioning from the field label, not from the rules	E5, E7	"Age 1 to 120" is a plausible human range. The real rule (18+) was never mapped. The spec was skimmed, not decomposed into conditions
RC3: Assuming invalid means one class	E2, E6, E8, E9	Invalid inputs fail in different ways (range, type, length, empty), each with its own error path
RC4: Treating the range as one object	E4, E7	Missed that thresholds inside a range split it into sub-classes
RC5: Confusing EP with sampling	E11, B1	Picking one convenient value per class turns a design technique into random testing
RC6: Not applying BVA after EP	B1 to B8	EP finds classes, BVA finds edges. Only the first step was done, and mid-values were used
RC7: Optimism bias (valid-path thinking)	B7	Invalid values were picked as "obviously wrong" (15, 75), not as "barely wrong"
RC8: No traceability to requirements	All	Without a requirement-to-class map, nobody can check whether a class is correct or complete

4. Correct Partitioning

4.1 Rules for a valid class

1. **Homogeneous behavior:** every member is accepted, or every member is rejected with the same error.
2. **Single reason:** an invalid class has one failure cause.
3. **Mutually exclusive and collectively exhaustive** across the whole input domain.
4. A threshold inside a range **splits** the range.

4.2 Corrected classes

Field	Valid classes	Invalid classes
Quantity	V1: 1 to 10	I1: ≤ 0 (zero, negative) · I2: ≥ 11 · I3: non-integer (2.5) · I4: non-numeric ("abc") · I5: blank · I6: special format ("1e1", "007", " 5 ")
Coupon %	V1: 0 to 50	I1: < 0 · I2: 51 to 99 · I3: ≥ 100 (check whether 100 is a special case) · I4: non-numeric · I5: blank
Age	V1: 18 to 120	I1: 1 to 17 (underage, blocked) · I2: ≤ 0 · I3: > 120 · I4: non-numeric · I5: blank
Free-shipping total	V1: below 999 (shipping charged) · V2: ≥ 999 (free)	I1: ≤ 0 or empty cart · I2: above the system maximum · I3: non-numeric
CVV	V1: exactly 3 digits	I1: fewer than 3 digits · I2: more than 3 digits · I3: non-digit characters · I4: blank
Pincode	V1: exactly 6 digits	I1: fewer than 6 · I2: more than 6 · I3: non-digit · I4: blank

Note that the free-shipping field has **two valid classes**, because the same valid input produces two different outcomes.

5. Corrected Test Cases (EP + BVA)

BVA convention: min−1, min, min+1, nominal, max−1, max, max+1. Nominal values are shown only where they add a class representative.

5.1 Quantity (valid 1 to 10)

ID	Input	Class	Expected
Q01	-1	I1	Rejected, “Enter a valid quantity”
Q02	0	I1 (min−1)	Rejected
Q03	1	V1 (min)	Accepted
Q04	2	V1 (min+1)	Accepted
Q05	5	V1 (nominal)	Accepted
Q06	9	V1 (max−1)	Accepted
Q07	10	V1 (max)	Accepted
Q08	11	I2 (max+1)	Rejected, “Maximum 10 per item”
Q09	2.5	I3	Rejected
Q10	“abc”	I4	Rejected
Q11	blank	I5	Rejected
Q12	“007”, “ 5 “, “1e1”	I6	Defined behavior (normalize or reject), no crash

5.2 Coupon (valid 0 to 50)

ID	Input	Class	Expected
C01	-1	I1 (min−1)	Rejected, price not increased
C02	0	V1 (min)	Accepted, price unchanged
C03	1	V1 (min+1)	Accepted
C04	25	V1 (nominal)	Accepted
C05	49	V1 (max−1)	Accepted
C06	50	V1 (max)	Accepted, half price
C07	51	I2 (max+1)	Rejected
C08	100	I3	Rejected, order not free
C09	“abc”, blank	I4, I5	Rejected

5.3 Age (valid 18 to 120)

ID	Input	Class	Expected
A01	-1 / 0	I2	Rejected
A02	17	I1 (min−1)	**Blocked** (underage)
A03	18	V1 (min)	Allowed
A04	19	V1 (min+1)	Allowed
A05	40	V1 (nominal)	Allowed
A06	119	V1 (max−1)	Allowed
A07	120	V1 (max)	Allowed
A08	121	I3 (max+1)	Rejected
A09	“abc”, blank	I4, I5	Rejected

5.4 Free-shipping threshold (₹999)

ID	Cart total	Class	Expected
S01	0 / empty	I1	Checkout blocked
S02	1	V1 (just above the minimum)	Shipping charged
S03	998	V1 (threshold−1)	Shipping charged
S04	998.99	V1 (threshold−0.01)	Shipping charged
S05	999	V2 (threshold)	**Free shipping**

S06	999.01	V2	Free shipping
S07	1,000	V2	Free shipping
S08	Above the max order value (+1)	I2	Rejected or flagged
S09	Cart at 1,100 with a coupon bringing it to 990	Interaction	Defined rule: is the threshold checked before or after the discount?

5.5 CVV (exactly 3 digits) and Pincode (exactly 6 digits)

ID	Input	Class	Expected
V01	CVV blank	I4	Rejected
V02	CVV 12	I1 (len−1)	Rejected
V03	CVV 123	V1	Accepted
V04	CVV 1234	I2 (len+1)	Rejected
V05	CVV 12a	I3	Rejected
V06	CVV “012” (leading zero)	V1	Accepted, zero not dropped
P01	Pincode 60001	I1 (len−1)	Rejected
P02	Pincode 600001	V1	Accepted
P03	Pincode 6000011	I2 (len+1)	Rejected
P04	Pincode 6000a1, blank	I3, I4	Rejected
P05	Pincode 012345	V1	Accepted, leading zero preserved

5.6 Combination checks (where valid and invalid inputs meet)

ID	Input	Expected
X01	Quantity 11 and coupon 51 together	Both errors shown, or defined precedence
X02	Age 17 with an alcohol item and quantity 1	Item blocked despite valid quantity
X03	Quantity 10 × price making the total exactly 999	Free shipping

Suite size: 12 + 9 + 9 + 9 + 11 + 3 = **53 tests**, against 8 in the flawed design.

6. Re-running the Seeded Defects

Seeded defect	Caught by	Result
Quantity >= 0	Q02	Caught
Quantity < 10	Q07	Caught
Coupon accepts -5%	C01	Caught
Age > 18	A03	Caught
Age 17 allowed	A02	Caught
Shipping > 999	S05	Caught
CVV accepts 4 digits	V04	Caught
Pincode accepts 5 digits	P01	Caught

8 of 8 caught, up from 0 of 8.

7. Why the Corrections Improve Defect Detection

1. **Pure classes make each test meaningful.** When every member of a class behaves the same, one test truly represents the rest. In the flawed design, one value stood for a mixed class, so a pass told nothing about the other half.
2. **Separate invalid classes expose separate error paths.** Each rejection reason (range, type, length, blank) is a different line of code, so each needs its own test. The lumped “invalid” class exercised one at best.
3. **BVA matches where programmers actually err.** Faults cluster at comparison operators (< vs <=) and off-by-one limits. Testing min−1, min, max, max+1 turns a likely defect into a certain failure, instead of relying on luck.

4. **Invalid values sit one step from the edge.** A rejected 11 proves the limit is enforced precisely. A rejected 75 only proves the check exists somewhere.
5. **Thresholds create new classes.** Splitting the shipping range at 999 turns a hidden decision into two tested outcomes.
6. **Traceability makes errors visible.** Mapping each class to a spec sentence lets a reviewer see that “age 1 to 120 valid” contradicts the 18+ rule.
7. **Cost stays low.** About 6 times the tests, but each is a few seconds of input, and the suite replaces open-ended exploratory guessing with a countable, reviewable set.

Effectiveness comparison

Measure	Flawed design	Corrected design
Tests	8	53
Valid classes covered correctly	partial	100%
Invalid classes covered	about 17%	100%
Boundary values exercised	0	about 30
Seeded boundary defects caught	0 of 8	8 of 8
Reviewable against requirements	No	Yes (class-to-rule map)

8. Review Checklist to Prevent Recurrence

Check	Question
Homogeneity	Would every member of this class get the same system response?
Single reason	Does each invalid class fail for exactly one reason?
Coverage	Do classes together cover every possible input, with no overlap?
Thresholds	Does any rule inside the range split it?
Types	Are non-numeric, blank, format and length cases listed?
Boundaries	Are min−1, min, max, max+1 present for each limit?
Traceability	Does every class cite a requirement?

9. Summary

Aspect	Finding
EP errors	11: mixed valid/invalid classes, wrong validity labels, classes straddling thresholds, missing and lumped invalid classes
BVA errors	8: no boundaries, invalid-side values far from edges, no format or length edges
Root causes	Typing by data type, skimming the spec, one-invalid-class assumption, EP treated as sampling, no BVA step, no traceability
Correction	Behavior-based classes, one reason per invalid class, thresholds as splits, seven-point BVA on every limit
Outcome	53 tests, 8 of 8 seeded defects caught versus 0 of 8, with every class traceable to a rule

2.A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary

conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.

Evaluating a Random-Testing Approach on a Banking Application

Assumption: no test log or coverage report came with your message, so I built a representative scenario. Send your real figures and I'll redo the numbers.

Spec assumed (the oracle):

Rule	Value
Transfer amount	₹1 to ₹1,00,000 per transaction, up to 2 decimals
Daily transfer cap	₹2,00,000 cumulative
Minimum balance retained	₹500
OTP required	amount above ₹10,000
Failed-PIN lockout	5th consecutive failure

1. The Approach Under Review

Item	What the engineer did
Method	Ad hoc and random inputs, with no design technique or documented cases
Volume	About 500 executions over two weeks
Claim	"High coverage", based on screens visited and features touched
Result	Critical defects reached UAT or production

ID	Escaped defect	Severity
D1	₹1,00,000 (exactly the limit) rejected, because the code uses >= where > is needed	Critical
D2	₹0 transfer accepted	High
D3	Daily cap enforced at ₹2,00,001, not ₹2,00,000	Critical
D4	OTP skipped at exactly ₹10,000	High
D5	Balance ₹500 minus ₹1 still allowed, so the balance drops below the minimum	High
D6	Cap not enforced across two sessions	Critical
D7	₹100.005 rounds inconsistently between debit and credit	Medium

2. Errors in Strategy

#	Error	Why it is an error
S1	No test design technique	Inputs came from habit and convenience, not from the rules in the spec
S2	No requirement traceability	Nobody can say which rule each test checks, so gaps cannot be seen
S3	No expected results defined in advance	Pass or fail was judged by "looks fine", so wrong-but-plausible results (D7) pass
S4	Uniform sampling of the input space	Defects sit at specific values, but random draws spread evenly across millions of values
S5	No risk prioritisation	Money-moving limits got the same effort as cosmetic screens
S6	No state or sequence design	The daily cap and lockout depend on history, and isolated random actions rarely build the right history
S7	No negative or invalid-class planning	Zero, negative, over-limit, non-numeric and blank inputs appear only by luck
S8	No repeatability	Random runs without a recorded seed cannot be re-run to confirm a fix

S9	No stopping criterion	"Enough" was a feeling, not a measure

3. Errors in Coverage Assumptions

#	Assumption	Why it fails
A1	"We visited every screen, so coverage is high"	That is feature or screen coverage. It says nothing about which values, branches or rules ran
A2	"500 tests is a lot"	Test count measures effort, not adequacy. 500 tests could all be the same equivalence class
A3	"Random means unbiased, so it covers everything"	Random is unbiased over the input space, but defects are heavily biased toward boundaries
A4	"Coverage was never measured, so it is probably fine"	An unmeasured quantity can't be claimed
A5	"No failures found means the module is good"	Absence of evidence is weak when the search was blind
A6	"Valid-looking inputs are representative"	Testers tend to pick realistic values (₹5,000), which avoid every edge

3.1 How likely is random testing to find a boundary defect?

Take amounts drawn uniformly from ₹1 to ₹10,00,000 (about 10^6 values). A defect that triggers only at exactly ₹1,00,000 (D1) has a hit chance of 1 in 10^6 per test.

Random tests run	Probability of hitting D1
500	$1 - (1 - 10^{-6})^{500} \approx$ **0.05%**
5,000	≈ 0.5%
1,00,000	≈ 9.5%
10,00,000	≈ 63%

A structured test lands on ₹1,00,000 with **certainty in one execution**. D3, D4 and D5 have the same property.

3.2 Defect detection by the random approach (as logged)

Defect type	Needs	Caught by random testing?
D1, D3, D4, D5 (exact-value edges)	Specific values	No, probability ≈ 0.05% each
D2 (zero)	A deliberate invalid value	Only by luck
D6 (cumulative cap)	A designed multi-step sequence	No
D7 (rounding)	Decimals at the 3rd place	No

Result: 0 of 7 critical defects were reliably catchable.

4. Root-Cause Analysis

Root cause	Defects it explains	Reasoning
RC1: Mistaking activity for adequacy	All	Visible effort (500 runs, all screens) was read as coverage, with no metric behind the claim
RC2: No model of the input domain	D1 to D5	Without partitioning, the tester never learned that ₹1,00,000, ₹10,000 and ₹2,00,000 are special
RC3: Ignoring where defects live	D1, D3, D4, D5	Programmer errors cluster at comparison operators (> vs >=) and off-by-one limits, which only boundary values expose

RC4: Stateless thinking about a stateful system	D3, D6	A cumulative limit needs a prior history, and random single actions don't build one
RC5: No oracle	D7	Results were not compared to a computed expected value
RC6: Optimistic test data	D1 to D5	People type plausible amounts, and plausible amounts sit mid-range
RC7: No risk ranking	All	Money limits are the highest-impact rules and got no extra scrutiny
RC8: Absence of a review gate	All	No one could challenge the claim, because it carried no evidence

5. Corrective Black-Box Techniques

Technique	What it adds	Defects targeted
Equivalence partitioning (EP)	Splits each field into valid and invalid classes with one reason per class, so every rule is represented	D2, D5, D7
Boundary value analysis (BVA)	Tests min−1, min, min+1, max−1, max, max+1 on every limit	D1, D3, D4, D5
Decision table	Enumerates combinations of conditions (balance, daily cap, OTP, amount) with their expected actions	Rule interactions, precedence
State transition testing	Walks through account and session states (daily total, failed-PIN counter, locked or active)	D3, D6, lockout
Use-case or scenario testing	End-to-end flows with expected results	Integration defects
Pairwise testing	Covers every pair of factors (channel × amount class × account type) with few tests	Combinations
Error guessing (structured)	A checklist from past banking defects: rounding, leading zeros, double submit	D7 and the like
Risk-based prioritisation	Orders everything by money impact and likelihood	Effort allocation
Boundary-biased random testing (as a supplement)	Random generation seeded with boundary values, with fixed seeds and a computed oracle	Residual unknowns

Random testing isn't useless. As the **only** technique it is inadequate, but as a seeded and oracle-checked supplement it adds exploration.

6. Reconstructed Test Design

6.1 Transfer amount (valid ₹1 to ₹1,00,000)

ID	Input	Class or BVA point	Expected
T01	−1	Invalid, negative	Rejected
T02	0	min−1	Rejected (catches D2)
T03	0.99	below min	Rejected
T04	1	min	Accepted
T05	1.01	min+1	Accepted
T06	5,000	nominal	Accepted, no OTP
T07	99,999	max−1	Accepted, OTP required
T08	**1,00,000**	max	**Accepted** (catches D1)
T09	1,00,001	max+1	Rejected

T10	100.005, 100.004	rounding edge	Debit equals credit, defined rounding (catches D7)
T11	“abc”, blank, “1e5”, “00100”	invalid format	Rejected or normalised, no crash

6.2 OTP threshold (above ₹10,000)

ID	Input	Expected
O01	9,999	No OTP
O02	**10,000**	No OTP (the rule is “above”, catches D4)
O03	10,001	OTP required
O04	10,001 with a wrong OTP	Rejected, no debit

6.3 Balance and minimum balance (₹500 retained)

ID	Balance	Amount	Expected
B01	1,000	500	Accepted, balance 500
B02	1,000	501	Rejected (catches D5)
B03	1,000	499	Accepted
B04	500	1	Rejected

6.4 Daily cap (₹2,00,000), stateful

ID	Sequence	Expected
C01	1,00,000 + 99,999	Both accepted
C02	1,00,000 + 1,00,000	Both accepted, total exactly 2,00,000
C03	1,00,000 + 1,00,000 + 1	Third rejected (catches D3)
C04	1,99,999 + 1, then + 1	Second accepted, third rejected
C05	Same sequence split across **two sessions or channels**	Cap shared (catches D6)
C06	Day rollover after the cap is reached	Counter resets

6.5 Lockout and state

ID	Sequence	Expected
L01	4 wrong PINs, then correct	Success, counter resets
L02	5 wrong PINs	Locked
L03	Correct PIN after lock	Still blocked
L04	Double-click on Submit	Exactly one debit

6.6 Decision table (combined rules)

Rule	Amount ≤ 1,00,000	Balance OK	Cap OK	> ₹10,000	Action
R1	N	any	any	any	Reject (limit)
R2	Y	N	any	any	Reject (balance)
R3	Y	Y	N	any	Reject (cap)
R4	Y	Y	Y	N	Execute, no OTP
R5	Y	Y	Y	Y	OTP, then execute

Pairs of failing conditions (for example over-limit and insufficient balance) are added to fix the error precedence.

Suite size: about 50 designed tests, each with a stated rule, input, expected result and traceability to the spec.

7. Re-running the Escaped Defects

Defect	Caught by	Random (500)	Structured
D1 limit edge	T08	≈ 0.05%	**Certain**
D2 zero accepted	T02	Luck	**Certain**
D3 daily cap off by one	C03	No	**Certain**
D4 OTP at 10,000	O02	≈ 0.05%	**Certain**
D5 minimum balance	B02	Luck	**Certain**
D6 cross-session cap	C05	No	**Certain**
D7 rounding	T10	No	**Certain**

8. Why These Techniques Give Better Adequacy and Reliability

1. **Adequacy becomes measurable.** Coverage is defined against something countable: classes, boundary points, decision-table rules, state transitions. “High coverage” turns from a feeling into a ratio, such as 50 of 50 planned rule elements exercised.
2. **Deterministic detection replaces probability.** BVA reaches each limit in one run. The random approach needed about 10^6 runs to reach a 63% chance on a single edge.
3. **Each test carries information.** EP ensures a passing test represents a whole class, so 50 tests stand for millions of inputs, while 500 random tests may represent a few classes.
4. **The technique matches the fault model.** BVA targets off-by-one and operator faults, decision tables target missed combinations, and state tests target history-dependent bugs. Each is aimed at a defect class that bank code actually has.
5. **Expected results are fixed first.** A written oracle makes wrong-but-plausible outputs (D7) detectable.
6. **Reproducibility.** Documented cases can be re-run after every fix and become regression tests. Random runs without seeds can’t confirm a fix.
7. **Traceability supports audit.** Banking is regulated, so each limit maps to a test and a result, which supports sign-off.
8. **Risk-ordered effort.** Limits, caps and OTP get the most scrutiny, so the budget lands where a failure costs most.
9. **Reliability improves cumulatively.** Every escaped defect becomes a permanent test, and the suite never forgets a past failure.

Effectiveness comparison

Measure	Random approach	Structured approach
Documented cases	0	about 50
Boundary values exercised	by chance	about 35 (all limits, two-sided)
Invalid classes covered	partial, accidental	all, by design
Stateful scenarios	none	10+
Expected results defined	no	yes
Critical seeded defects caught	0 of 7 reliably	7 of 7
Reproducible	no	yes
Coverage claim verifiable	no	yes

9. Recommended Operating Model

Layer	Practice
Design	EP → BVA → decision table → state model, from the spec
Execution	Risk-ordered, with expected values fixed beforehand
Supplement	Boundary-seeded random or property-based tests, fixed seeds, oracle-checked
Measurement	Track class, boundary, rule and transition coverage, plus code coverage as a second view
Quality gate	No release claim without a coverage table and zero open critical defects
Feedback	Every escaped defect becomes a regression test and updates the checklist

10. Summary

Aspect	Finding
Approach	Unstructured random testing, with coverage claimed from screens visited
Strategy errors	No technique, no oracle, no traceability, no risk ranking, no state design, no stopping criterion
Coverage errors	Feature coverage mistaken for test adequacy, and test count mistaken for coverage
Quantified weakness	About 0.05% chance per boundary defect with 500 random tests
Root causes	No domain model, defects clustered at edges, stateless thinking, optimistic data, no review gate
Corrective techniques	EP, BVA, decision table, state transition, pairwise, structured error guessing, risk-based ordering, with seeded random as a supplement
Outcome	About 50 traceable tests catch all 7 escaped defects, and adequacy becomes something you can measure

3.A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Auditing an Incomplete Control Flow Graph and Its Path Testing

Assumption: no code or CFG was attached, so I built a representative flawed example. Send your real code and diagram and I'll audit them line by line. The error taxonomy and the correction method apply to any module.

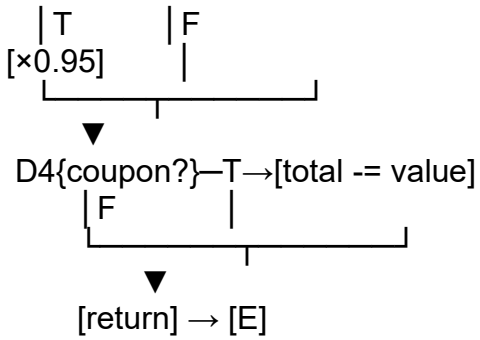
1. The Module Under Analysis

A bulk order-discount routine:

```
 1  applyDiscount(items[], member, coupon):
 2    total = 0
 3    for item in items:                        // L1
 4      if item.qty <= 0: continue              // D1
 5      price = item.price * item.qty
 6      if item.category == "ELEC" and item.qty > 5:   // D2 (compound)
 7          price *= 0.90
 8      total += price
 9    if member == GOLD or total > 10000:       // D3 (compound)
10      total *= 0.95
11    if coupon != null:                        // D4
12      while coupon.uses < coupon.max and total > 0:  // L2 (compound)
13        total -= coupon.value
14        coupon.uses += 1
15    if total < 0: total = 0                   // D5
16    return total
```

2. The Developer's Flawed CFG

[S] → [total=0] → [price=..., total+=price] → D3{member or total>10000}



Resulting figures claimed: 6 nodes, 1 decision-style structure, 4 “paths”, and 100% path coverage from 4 tests.

3. Errors in the CFG

#	Error	Detail	Type
E1	**Loop L1 missing**	The for over items is drawn as straight-line code. No back edge, no loop-exit decision	Missing loop
E2	**Loop L2 missing**	The coupon while is drawn as a single statement total -= value	Missing loop
E3	**Decision D1 missing**	The continue skip for qty ≤ 0 is absent, so the edge that bypasses lines 5 to 8 does not exist	Missing decision and edge
E4	**Decision D2 missing**	The ELEC discount branch is absent, so line 7 is never a node	Missing decision
E5	**Decision D5 missing**	The floor at zero is absent	Missing decision
E6	**Compound conditions drawn as single nodes**	D3 shows member or total>10000 as one diamond. Short-circuit evaluation makes it two predicates	Wrong granularity
E7	**Compound conditions not expanded**	D2 and the L2 condition (uses < max and total > 0) are also each two predicates	Wrong granularity
E8	**continue edge missing**	Jump-style control flow (continue, break, early return) creates edges that sequential reading overlooks	Missing edge
E9	**Merge nodes missing**	After each if, both branches must rejoin at a defined node	Structural flaw
E10	**No distinct exit node for the loop**	The edge from “no more items” to the next statement is absent	Missing edge

3.1 Errors in the metrics derived from the flawed graph

Metric	Developer’s value	Correct value
Predicate nodes	2 (D3, D4)	**10** (see section 4)
Cyclomatic complexity V(G)	3	**11**
Independent paths claimed	4	**11 basis paths**
Statement coverage claimed	100%	The loop bodies and the zero floor are untested

3.2 Errors in the path testing approach

#	Error	Effect

P1	**Path count derived from decisions seen in the graph, not from the code**	The graph hid 8 of 10 predicates, so the path count was 4 instead of 11
P2	**Loops tested only once**	Zero, one, many and max iterations never separated
P3	**Compound conditions tested as units**	Each part of A or B never shown to matter on its own
P4	**"100% path coverage" claimed**	Only 4 paths were tried, never reconciled against the code's actual count
P5	**No feasibility analysis**	Infeasible combinations were not identified, so the target was undefined
P6	**No expected-result oracle per path**	Paths were run but outputs not verified against computed values
P7	**Graph never reconciled to the code**	No step compared nodes and edges to the source lines

4. Root-Cause Analysis

Root cause	Errors it explains	Reasoning
RC1: Drawing the graph from memory or the "main idea", not line by line	E1 to E5	The graph showed what the function is for (discount, then coupon), not what the code does. Every control keyword (for, if, while, continue) must become a node
RC2: Treating loops as ordinary statements	E1, E2, E10, P2	A loop is a decision plus a back edge. Calling it "a step" erases both
RC3: Ignoring short-circuit evaluation	E6, E7, P3	A and B compiles to two branches. One diamond hides a whole outcome
RC4: Missed non-sequential jumps	E3, E8	continue, break and early return create edges that don't follow the textual order
RC5: Using test count as a goal	P4	Four tests passed, so "complete" was declared without a calculation to compare against
RC6: No cyclomatic complexity cross-check	P1, P7	V(G) computed three ways (edges, predicates, regions) would have disagreed with the drawing and flagged the error
RC7: No peer review of the graph	All	A graph that is never compared with the source cannot be challenged
RC8: Confusing coverage criteria	P3, P4	Statement, branch, condition and path coverage were treated as one thing

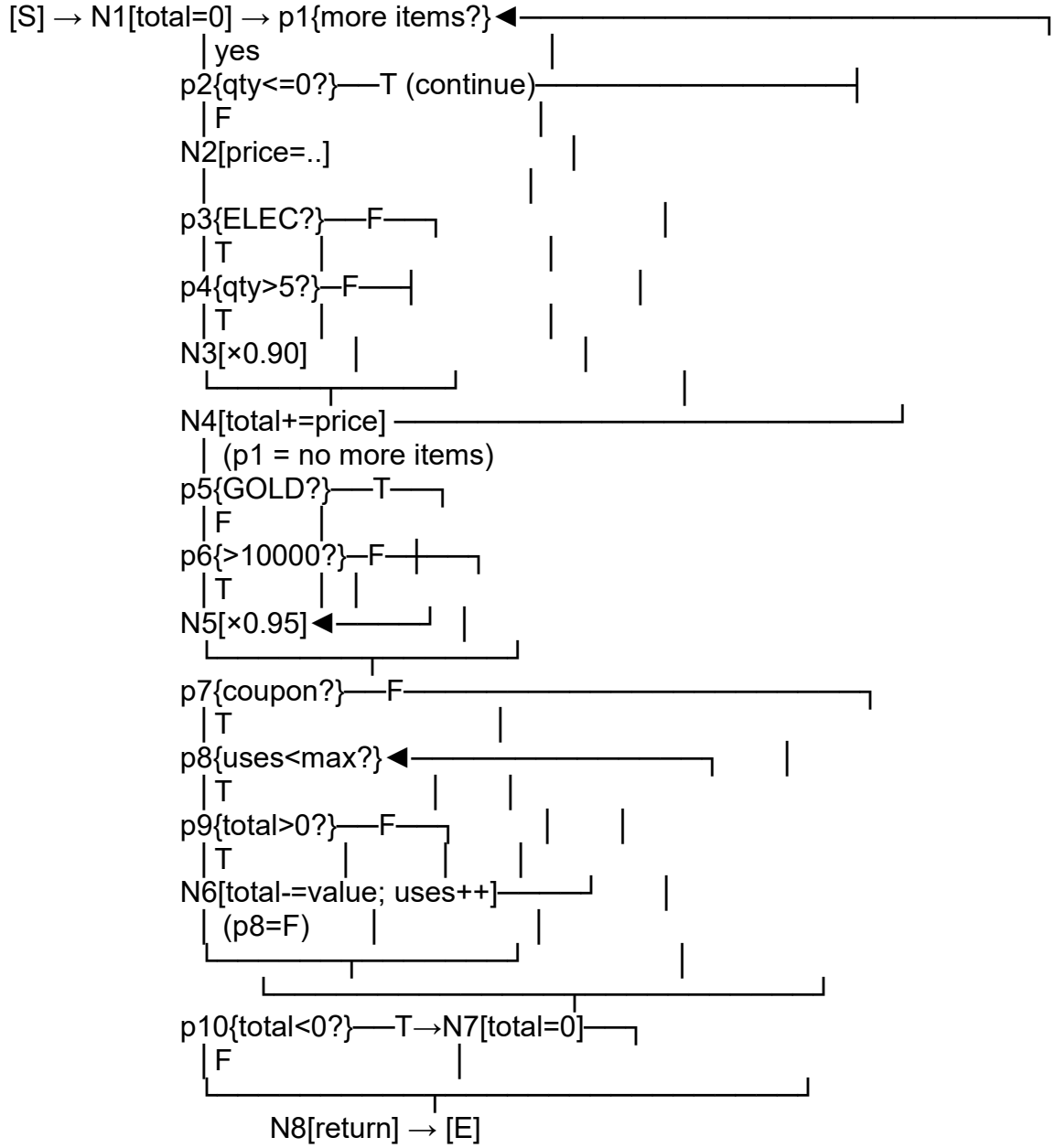
5. The Corrected Control Flow Representation

5.1 Predicate inventory (short-circuit expanded)

ID	Predicate	Source
p1	more items? (loop test)	L1
p2	item.qty <= 0	D1
p3	category == ELEC	D2, part 1
p4	qty > 5	D2, part 2 (evaluated only if p3 is true)
p5	member == GOLD	D3, part 1
p6	total > 10000	D3, part 2 (evaluated only if p5 is false)
p7	coupon != null	D4
p8	coupon.uses < coupon.max	L2, part 1
p9	total > 0	L2, part 2 (evaluated only if p8 is true)
p10	total < 0	D5

10 predicate nodes.

5.2 Corrected graph



5.3 Complexity (three independent checks)

Method	Calculation	V(G)
Predicates + 1	10 + 1	**11**
Edges − nodes + 2	29 − 20 + 2	**11**
Bounded regions	10 enclosed + 1 outer	**11**

All three agree, so the graph is internally consistent. The developer's graph gave 3, which would have failed this check.

6. Independent Paths

6.1 Basis set (11 paths)

Baseline B0: one valid non-ELEC item, not GOLD, total ≤ 10,000, no coupon, total ≥ 0.

Path	Variation from B0	Test input	Expected
B0	Baseline	1 item, ₹500 × 2, non-member, no coupon	1,000
B1	p1 false at once (empty list)	items = []	0
B2	p2 true (skip)	1 item with qty 0	0 (item skipped)
B3	p3 true, p4 false	ELEC, qty 5	no 10% discount
B4	p3 true, p4 true	ELEC, qty 6, ₹1,000	6,000 × 0.90 = 5,400
B5	p5 true	GOLD, small total	total × 0.95
B6	p5 false, p6 true	non-member, total 12,000	11,400
B7	p7 true, p8 false	coupon with uses = max	no coupon applied
B8	p8 true, p9 false	coupon, total already 0	loop exits via p9
B9	p8 true, p9 true, loop body once	coupon ₹100, max 1	total − 100
B10	p10 true	coupon larger than the total	total floored to 0

6.2 Loop tests (extend the basis set)

Loop	Test	Input	Observe
L1	0 iterations	empty list	B1
L1	1 iteration	1 item	B0
L1	2 iterations	2 items	accumulation correct
L1	typical (n)	10 items	sum correct
L1	max (assumed 100)	100 items	no overflow or timeout
L1	mixed skip and keep	qty 0, 2, 0, 3	continue does not break the loop
L2	0 iterations	uses = max	B7
L2	1 iteration	max = 1	B9
L2	2 iterations	max = 2, total large	both uses consumed
L2	exits on total	max = 10, total reaches 0 after 3	stops on p9, not p8
L2	exits on max	max = 2, total large	stops on p8, not p9

6.3 Condition coverage for compound predicates

Predicate	Condition varied alone	Pair
D2 ELEC and qty>5	ELEC	(ELEC, 6) vs (BOOKS, 6)
D2	qty > 5	(ELEC, 6) vs (ELEC, 5)
D3 GOLD or total>10000	GOLD	(GOLD, 5,000) vs (non-member, 5,000)
D3	total > 10000	(non-member, 10,001) vs (non-member, 10,000)
L2 uses<max and total>0	uses < max	(0, 1, total 100) vs (1, 1, total 100)
L2	total > 0	(uses 0, max 1, total 1) vs (uses 0, max 1, total 0)

6.4 Feasibility

The theoretical path count explodes with loops (every combination of per-item outcomes times every coupon iteration count), so **full path coverage is infinite or impractical** for this module. The correct target is:

Level	Target	Achievable
Basis paths	11 of 11	Yes
Branch (every edge)	100%	Yes
Condition / MC/DC	all 10 predicates	Yes
Loop classes (0, 1, 2, n, max)	both loops	Yes
All paths	unbounded	No, so a stated and justified substitute is required

Infeasible combinations are noted, not tested: for example, B8 (loop exits via p9 on entry) requires a total of 0, which is only reachable when all items are skipped or the earlier discounts zero it, so it needs a constructed input rather than a natural one.

7. Coverage Comparison

Criterion	Flawed approach (4 tests)	Corrected approach (about 25 tests)
Statement	about 60% (lines 5 to 7, 13, 14 and 15 unexecuted)	**100%**
Branch / edge	about 35% (7 of 20 outcomes)	**100%**
Predicates drawn	2 of 10	**10 of 10**
Condition (atomic T and F)	2 of 10 conditions	**10 of 10**
Basis paths	3 of 11	**11 of 11**
Loop classes	0 of 10	**10 of 10**
V(G) reconciled with the code	No	Yes, three methods agree

8. Defect Detection: Seeded Faults

Seeded fault	Caught by flawed tests?	Caught by corrected tests	By
qty <= 0 written as qty < 0 (zero-qty item not skipped)	No	Yes	B2
ELEC discount applied at qty >= 5	No	Yes	B3
or and and swapped in D3	No	Yes	D3 condition pairs
total > 10000 written as >= 10000	No	Yes	pair at 10,000 / 10,001
Coupon loop never exits (missing uses += 1)	No (loop unreached)	Yes	L2 tests (hang detected by timeout)
Loop runs max + 1 times (off by one)	No	Yes	L2 exits-on-max
Negative total returned	No	Yes	B10
continue replaced by break	No	Yes	L1 mixed skip and keep
Discount applied in the wrong order (coupon before percent)	No	Yes	B6 combined with B9

9 of 9 caught, up from 0 of 9.

9. Why Better Path Testing Ensures Better Coverage

1. **The graph becomes the test requirement.** Every missing decision removes tests from the plan, so an accurate graph is the precondition for any coverage claim. A graph with 2 predicates asks for 4 tests, while the real code asks for 11 or more.

2. **Cyclomatic complexity gives an independent check.** If the node and edge count, the predicate count and the region count disagree, the drawing is wrong. This caught the error mechanically, with no deep code reading.
3. **Basis paths guarantee each edge is exercised.** Any path through the graph is a combination of the 11 basis paths, so a set that includes them covers every statement and every branch.
4. **Short-circuit expansion exposes hidden outcomes.** Splitting A or B into two predicates creates the case "A false, B evaluated" that single-diamond testing skips, which is where or/and swaps and boundary faults sit.
5. **Loop tests reach distinct faults.** Zero iterations find missing-initialization and empty-input faults. One iteration finds off-by-one faults. Many iterations find accumulation errors. The max case finds overflow and timeout. A single pass through each loop finds almost none of these.
6. **Realistic targets replace false claims.** Saying "11 basis paths plus loop classes plus MC/DC" is verifiable. Saying "100% paths" for a module with two loops is impossible and was therefore a sign of an error.
7. **Review becomes possible.** A complete graph with numbered predicates and a path-to-test table can be checked by another person against the source, which closes the loop that RC7 left open.

10. Checklist to Prevent Recurrence

Check	Question
Keyword scan	Does every if, else if, for, while, case, catch, continue, break and return have a node or edge?
Short circuit	Is every and, or, ?: split into separate predicates?
Loops	Does every loop have a test node, a body, a back edge and an exit edge?
Complexity	Do edges − nodes + 2, predicates + 1 and regions all give the same V(G)?
Reconciliation	Does every source line map to a node, and every node to a source line?
Paths	Is each basis path linked to one test with a computed expected value?
Feasibility	Are infeasible paths listed with the reason, and loops bounded by a stated criterion?
Review	Has someone other than the author compared the graph to the code?

11. Summary

Aspect	Finding
Graph errors	10: two loops missing, three decisions missing, compound conditions drawn as single nodes, jump and exit edges absent
Metric errors	2 predicates and V(G) = 3 claimed, against 10 predicates and V(G) = 11 in the code
Approach errors	Path count taken from the drawing, loops tested once, compound conditions unsplit, "100% path coverage" claimed without a basis
Root causes	Graph drawn from intent rather than code, loops treated as statements, short-circuit ignored, no complexity cross-check, no review
Correction	Predicate inventory, expanded graph with 20 nodes and 29 edges, V(G) = 11 verified three ways, 11 basis paths, loop-class and MC/DC tests
Outcome	About 25 tests reach 100% statement, branch, condition and basis-path coverage and catch 9 of 9 seeded faults, up from 0 of 9

4.A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

Why 100% Statement Coverage Missed Logical Errors in Conditionals

Assumption: no code or coverage report came with your message, so I built a representative module. Send your real code and I'll redo the counts. The reasoning applies to any module with conditionals.

1. The Module Under Analysis

A loan eligibility routine:

```
 1  checkLoan(age, income, creditScore, existingLoans):
 2    status = "REJECT"
 3    if (age >= 21 and age <= 60)                    // D1
 4       if (income >= 30000 or creditScore >= 750)    // D2
 5          status = "APPROVE"
 6    if (existingLoans > 2)                          // D3
 7       status = "REVIEW"
 8    if (status == "APPROVE" and creditScore < 600)    // D4 (contradictory, see below)
 9       status = "REJECT"
10    return status
```

Four decisions, three compound. Atomic conditions: a1 (age ≥ 21), a2 (age ≤ 60), b1 (income ≥ 30000), b2 (score ≥ 750), c1 (loans > 2), d1 (status == APPROVE), d2 (score < 600). That is 7.

2. What the Team Did

2.1 The suite that achieved 100% statement coverage

Test	age	income	score	loans	Result
T1	35	50,000	780	3	REVIEW
T2	18	10,000	500	0	REJECT

T1 runs lines 2 to 7 and 10. T2 runs lines 2, 3, 6 and 10. Line 9 never executes in either test... so to reach 100% the team must add a third test, T3 (age 35, income 50,000, score 550, loans 0 → APPROVE then REJECT), which covers lines 8 and 9. Line 9 is reachable only when income is high and the score is below 600, a combination the spec may or may not allow.

Three tests, every line executed, 100% statement coverage.

2.2 What that figure actually proves

Statement coverage answers one question: *did each line run at least once?* It does not ask whether a decision went both ways, whether each part of a compound condition mattered, or whether the output was correct.

3. Evaluating Adequacy

Criterion	Achieved by T1 to T3	Total	Result
Statement	8	8	**100%**
Branch outcomes	D1 T,F; D2 T (T1,T3), F (T2 never reaches D2 with D1 true); D3 T,F; D4 T,F	8	**7 of 8 (87.5%)**
Condition (atomic T and F)	about 8 of 14	14	**about 57%**
MC/DC	0 of 7 conditions shown independent	7	**0%**
Basis paths	3	8	**38%**

So the module is adequately tested only by the weakest criterion. Branch coverage is already incomplete: D2 never takes its false outcome, because T2 fails D1 and never reaches line 4.

4. Gaps in the Coverage Criteria

#	Gap	Explanation
G1	**Implicit else branches ignored**	An if without an else has a false edge that runs no statement, so statement coverage never needs it. D2's false outcome (income low and score low) is never forced
G2	**Compound conditions untested internally**	a1 and a2 is satisfied by one pair of values. The operator and versus or is never distinguished
G3	**Boundary operators unprobed**	>= versus > and <= versus < need values exactly on 21, 60, 30000, 750
G4	**Condition interactions**	D3 overwrites D1/D2's result, so ordering matters, but no test varies both
G5	**Dead or contradictory logic undetected**	D4 requires status APPROVE and score < 600, but APPROVE needs income ≥ 30000 **or** score ≥ 750. Only high income with a low score reaches it. Without a path analysis, this reachability question is never asked
G6	**No oracle check**	Coverage measures execution, not correctness. A line that ran and returned a wrong value still counts as covered
G7	**Short-circuit evaluation hidden**	b1 or b2 never evaluates b2 when b1 is true, so b2 may never be exercised even though the line is covered

5. Why Statement Coverage Alone Was Insufficient

5.1 Seeded logical errors that survive 100% statement coverage

Mutant	Change	Caught by T1 to T3?	Why not
M1	D1: and becomes or	**No**	T1 and T3 have both parts true, and T2 has both false, so and and or agree
M2	D1: age >= 21 becomes age > 21	**No**	No test uses age 21
M3	D1: age <= 60 becomes age < 60	**No**	No test uses 60
M4	D2: or becomes and	**No**	T1 has both true, T2 never reaches D2
M5	D2: income >= 30000 becomes > 30000	**No**	No test at exactly 30,000
M6	D3: loans > 2 becomes >= 2	**No**	No test with exactly 2 loans
M7	D2 false branch returns APPROVE (condition inverted for low income and low score)	**No**	That combination is never run
M8	D4: score < 600 becomes score <= 600	**No**	No test at exactly 600

0 of 8 caught. Every one is a pure logic error: the same statements execute, only the decision differs.

5.2 The structural reasons

1. **Statements are nodes, but defects live on edges.** A decision is an edge-choice, and a wrong condition changes which edge is taken without changing which nodes exist.
2. **One test can cover many statements.** T1 alone covers six lines, so the metric rewards breadth along a single path, not variety of decisions.
3. **The metric is monotone in the wrong place.** Adding a test that re-runs covered lines raises nothing, while adding a test that flips a hidden condition also raises nothing, so the number gives no signal to continue.

4. **A passing percentage can't show an absent test.** 100% hides that D2 false, age 21, age 60 and income 30,000 were never exercised.

5.3 Criteria hierarchy

Path coverage (strongest, can be unbounded)
▲
Multiple condition coverage (all 2^n combinations)
▲
MC/DC (each condition shown to affect the outcome)
▲
Condition/decision coverage
▲
Branch (decision) coverage
▲
Statement coverage ← what was achieved

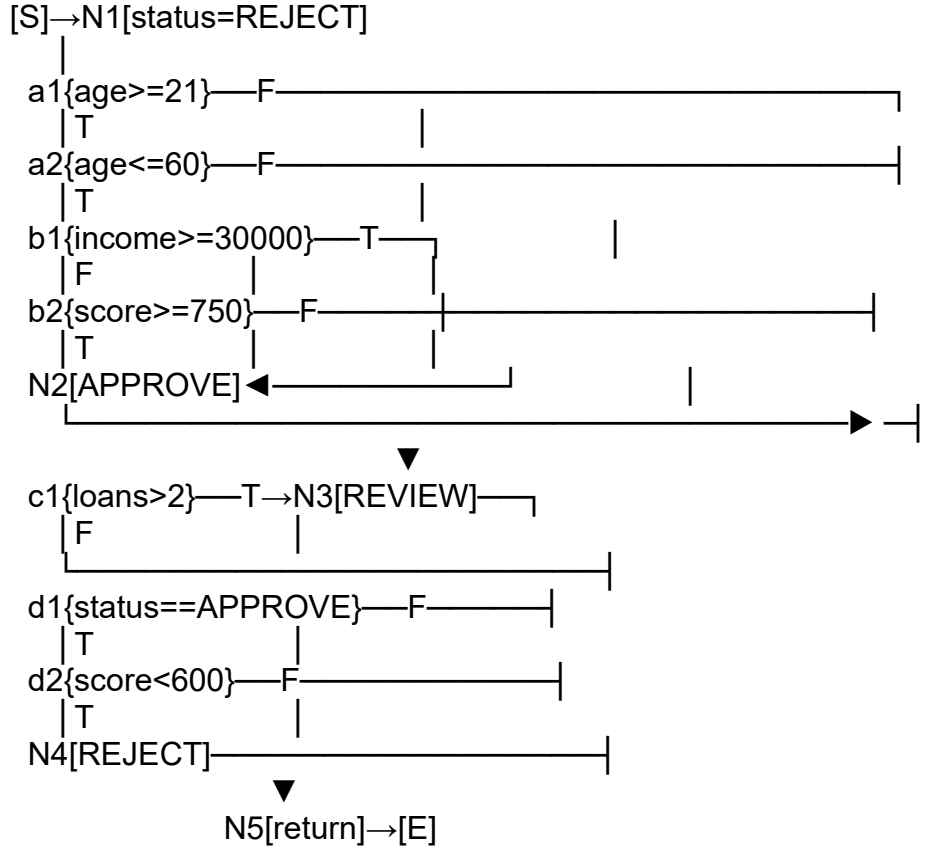
Statement coverage is the bottom rung and is subsumed by every other criterion, while the reverse is false.

6. Corrected Control Flow Representation

Short-circuit expansion turns each compound decision into separate predicates.

Decision	Expanded predicates
D1	a1; if true, a2
D2	b1; if false, b2
D3	c1
D4	d1; if true, d2

Predicate nodes: 2 + 2 + 1 + 2 = **7**. Cyclomatic complexity V(G) = 7 + 1 = **8**.



7. Recommended White-Box Strategy

Four layers, each closing a specific gap.

Layer 1: Branch (decision) coverage

Every decision takes both outcomes.

ID	age	income	score	loans	Outcomes forced	Expected
B1	35	50,000	780	0	D1 T, D2 T, D3 F, D4 F (d2 F)	APPROVE
B2	18	50,000	780	0	D1 F	REJECT
B3	35	10,000	500	0	D2 F	REJECT
B4	35	50,000	780	3	D3 T	REVIEW
B5	35	50,000	550	0	D4 T	REJECT

Layer 2: MC/DC on every compound condition

Each atomic condition flips the decision on its own, with the others held fixed.

Decision	Condition	Test pair	Outcome flips
D1 a1 and a2	a1	age 25 vs 18 (a2 true)	T ↔ F
D1	a2	age 25 vs 70 (a1 true)	T ↔ F
D2 b1 or b2	b1	income 50,000 vs 10,000 (score 500)	T ↔ F
D2	b2	score 780 vs 500 (income 10,000)	T ↔ F
D4 d1 and d2	d1	status APPROVE vs not (score 550)	T ↔ F
D4	d2	score 550 vs 650 (status APPROVE)	T ↔ F

Minimum tests per decision of n conditions: n + 1.

Layer 3: Boundary tests on every relational operator

ID	Target	Inputs	Expected
R1	age >= 21	20 / 21 / 22	reject / eligible / eligible
R2	age <= 60	59 / 60 / 61	eligible / eligible / reject
R3	income >= 30000	29,999 / 30,000 / 30,001 (score 500)	reject / approve / approve
R4	score >= 750	749 / 750 / 751 (income 10,000)	reject / approve / approve
R5	loans > 2	2 / 3	no review / review
R6	score < 600	599 / 600 / 601 (status APPROVE)	reject / approve / approve

Layer 4: Path coverage

The module has no loops, so all paths are enumerable. Theoretical count is $2^4 = 16$, but the real count is higher once short-circuit splits are counted, and some are infeasible. Basis paths (8) are the mandatory minimum, and the feasible full paths are added for the highest-risk interactions, namely D3 overwriting an APPROVE and D4 reversing one.

Path	Route	Input	Expected
P1	a1 F	18, any	REJECT
P2	a1 T, a2 F	70	REJECT
P3	a1 T, a2 T, b1 T	35, 50,000, 700	APPROVE
P4	b1 F, b2 T	35, 10,000, 780	APPROVE
P5	b1 F, b2 F	35, 10,000, 500	REJECT
P6	APPROVE then loans > 2	35, 50,000, 780, 3	REVIEW (D3 overrides)
P7	APPROVE, score < 600	35, 50,000, 550, 0	REJECT
P8	REJECT, loans > 2	18, 10,000, 500, 3	REVIEW

P8 is notable: a rejected applicant with many loans ends as REVIEW, not REJECT. Whether that is intended is a specification question that only an oracle-driven path test would surface.

Layer 5: Static check for infeasible and contradictory paths

D4 requires score < 600 and status APPROVE. APPROVE requires income ≥ 30000 **or** score ≥ 750, so the only route is high income with a low score, which P7 covers. If the specification said "score below 600 never approved", then D2 should have contained the rule and D4 is redundant logic, a design smell the path analysis exposes.

8. Coverage After the Revision

Criterion	Original (T1 to T3)	Revised (about 28 tests)
Statement	100%	100%
Branch	87.5%	100%
Condition	about 57%	100%
MC/DC	0 of 7	7 of 7
Basis paths	3 of 8	8 of 8
Boundary operators	0 of 7	7 of 7

9. Seeded Mutants, Re-run

Mutant	Caught by	Result
M1 and to or in D1	MC/DC pair on a1 and a2	Caught
M2 age > 21	R1 (age 21)	Caught
M3 age < 60	R2 (age 60)	Caught
M4 or to and in D2	MC/DC pairs b1, b2	Caught
M5 income > 30000	R3 (30,000)	Caught
M6 loans >= 2	R5 (loans 2)	Caught
M7 inverted D2 false branch	B3, P5	Caught
M8 score <= 600	R6 (600)	Caught

8 of 8 caught, up from 0 of 8.

10. Why the Revised Approach Is More Effective

1. **It tests decisions, not just lines.** Branch coverage forces each if both ways, which includes the implicit else that statement coverage ignores (G1).
2. **MC/DC proves each condition matters.** If changing only income >= 30000 changes the result, a fault in that comparison produces a wrong output. Without the pair, and and or look identical (M1, M4).
3. **Boundary tests target the commonest logic fault.** Errors cluster at < versus <=, so testing values on and beside each limit turns a probable defect into a certain failure.
4. **Path tests catch interaction faults.** D3 overwriting D2's result, and D4 reversing it, are visible only as complete routes (P6, P7, P8).
5. **Complexity gives a stopping rule.** V(G) = 8 sets a floor of 8 independent paths, so "enough" becomes a number and not a feeling.
6. **Mutants provide independent evidence.** Seeding faults and counting kills shows whether coverage translates into detection, which a coverage percentage alone never does.
7. **Costs stay moderate.** About 28 tests, each a single function call with computed expected output, against 3 tests that gave false confidence.
8. **A lower layer never replaces a higher one.** Each criterion subsumes the one beneath it, so adding branch, MC/DC and path never reduces what statement coverage gave.

Effectiveness comparison

Measure	Statement-only	Revised
Tests	3	about 28
Decisions both ways	7 of 8 outcomes	8 of 8
Atomic conditions shown independent	0	7
Boundary operators probed	0	7
Seeded logic faults caught	0 of 8	8 of 8

Specification questions surfaced (P8, D4)	0	2

11. Limits of the Analysis

1. **Coverage is not correctness.** Even 100% MC/DC is only as good as the expected values, so each test needs an oracle derived from the specification.
2. **Missing code is invisible.** White-box criteria cannot detect a rule that was never implemented, which is why black-box techniques (EP, BVA, decision tables) must run alongside.
3. **Loops change the picture.** This module has none. With loops, full path coverage becomes unbounded and the plan falls back to basis paths plus loop-boundary tests.
4. **Mutation score complements coverage.** Eight mutants is a small sample, so a larger automated mutation run is the stronger evidence.

12. Summary

Aspect	Finding
Claim	100% statement coverage with 3 tests
Real adequacy	Branch 87.5%, condition about 57%, MC/DC 0%, basis paths 3 of 8
Core flaw	Statement coverage measures that lines ran, not that decisions were right
Gaps	Implicit else edges, compound conditions, boundaries, condition interactions, short-circuit hiding, no oracle
Strategy	Branch + MC/DC + boundary + basis-path coverage, with a static check for contradictory logic
Outcome	About 28 tests, all 8 logic mutants caught versus 0 of 8, and two specification questions surfaced

5.A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

Auditing a Two-Class Partitioning of an Age Field (0 to 100)

Assumption: no tester document was attached, so I've reconstructed it from your description. **Spec assumed:** the age field accepts whole numbers from 0 to 100 inclusive, and anything else is rejected with an error.

1. The Flawed Design

Class	Definition	Representative test	Expected (tester's view)
Class 1	Negative numbers (< 0)	-5	Rejected
Class 2	Positive numbers (> 0)	25	Accepted

Two tests, no boundary values.

2. Errors in the Partitioning

#	Error	Why it is wrong
E1	**Class 2 mixes valid and invalid inputs.** "Positive" includes 1 to 100 (valid) and 101, 500, 99999 (invalid)	Members of one class must behave identically. 25 is accepted and 150 is rejected, so a pass on 25 says nothing about 150
E2	**Zero belongs to neither class.** Zero is neither negative nor positive, yet it is a valid age	The lowest valid value is left outside the model
E3	**The upper limit is invisible.** Nothing in the design is aware that 100 is a limit	A missing age <= 100 check would never be found
E4	**Partitioned by sign, not by behavior.** Sign is a mathematical property, but the system only cares about valid versus invalid	Root of E1 to E3
E5	**Missing invalid classes:** non-numeric ("abc"), blank, decimals (25.5), special characters, whitespace, very large numbers, leading zeros ("007")	Each follows a separate error path in the code
E6	**Class 1 is correct but incomplete.** Negatives are invalid, but extremely negative values (-2^{31}) can overflow	Possible sub-partition
E7	**One representative per class** (-5 and 25)	Even for correct classes, mid-range values are the least likely to reveal faults

Boundary value analysis errors

#	Error	Effect
B1	No BVA at all	Values -1, 0, 1, 99, 100, 101 were never tried
B2	Lower edge untested	age > 0 instead of age >= 0 would reject a newborn and pass unnoticed
B3	Upper edge untested	age < 100 or age <= 101 slips through
B4	Invalid side chosen far from the edge (-5)	-5 passes even when the check is age < -1

3. Root Causes

Root cause	Explanation
Partitioning by data property rather than system behavior	"Negative or positive" describes the number, not how the validator treats it
Ignoring the specification's limits	The range 0 to 100 defines three zones (below, inside, above), but only two were modeled

Assuming "invalid" means "negative"	Over-limit and non-numeric inputs were never considered
Treating EP as the whole technique	EP finds classes, and BVA is the required second step for the edges
No completeness check	The classes do not cover the full domain, since zero and everything non-numeric fall outside them

4. Corrected Partitions

Rules for a valid class: all members behave the same, an invalid class has one failure reason, and the classes together cover the whole domain without overlap.

ID	Class	Range / description	Expected
V1	Valid age	Integers 0 to 100	Accepted
I1	Below minimum	Integers < 0	Rejected, "Age cannot be negative"
I2	Above maximum	Integers > 100	Rejected, "Age must not exceed 100"
I3	Non-integer numeric	25.5, 0.1	Rejected, or defined rounding
I4	Non-numeric	"abc", "twenty"	Rejected
I5	Blank or whitespace	"", " "	Rejected as required
I6	Special characters	"@#", "1;2", SQL-style strings	Rejected, no error or crash
I7	Format variants	"007", "+25", "1e1", " 25 "	Documented behavior
I8	Extreme magnitude	2^{31}, -2^{31}, 99999999999	Rejected, no overflow

Optional split of V1 if the application treats minors, adults and seniors differently, for example 0 to 17, 18 to 64, 65 to 100. A threshold inside a range always creates new classes.

5. Boundary Value Analysis

Seven-point BVA on both limits:

Boundary	Values	Expected
Lower	-1, 0, 1	Rejected, accepted, accepted
Nominal	50	Accepted
Upper	99, 100, 101	Accepted, accepted, rejected

6. Corrected Test Cases

ID	Input	Class / point	Expected
T01	-2147483649	I8	Rejected, no overflow
T02	-5	I1	Rejected
T03	**-1**	I1, min−1	Rejected
T04	**0**	V1, min	Accepted
T05	**1**	V1, min+1	Accepted
T06	50	V1, nominal	Accepted
T07	**99**	V1, max−1	Accepted
T08	**100**	V1, max	Accepted
T09	**101**	I2, max+1	Rejected
T10	500	I2	Rejected
T11	25.5	I3	Rejected or defined
T12	"abc"	I4	Rejected
T13	blank / spaces	I5	Rejected
T14	"@#$", ' OR 1=1	I6	Rejected, no crash
T15	"007", " 25 ", "+25"	I7	Defined behavior
T16	99999999999	I8	Rejected

Total: **16 tests** versus 2.

7. Defects Detected

Seeded fault	Original (-5, 25)	Corrected	Caught by
age > 0 (rejects 0)	No	Yes	T04
age >= -1 (accepts -1)	No	Yes	T03
Upper check missing	No	Yes	T09, T10
age < 100 (rejects 100)	No	Yes	T08
age <= 101	No	Yes	T09
Non-numeric causes a crash	No	Yes	T12
Decimal truncated silently	No	Yes	T11
Overflow on huge values	No	Yes	T01, T16

8 of 8 caught, up from 0 of 8.

8. Why This Ensures Robust Validation

1. **Complete domain coverage.** The classes now cover negatives, zero, in-range, above range, and non-numeric inputs, so no input falls between classes.
2. **Homogeneous classes make one test representative.** Because valid and invalid inputs no longer share a class, a pass or fail carries real meaning.
3. **BVA targets where faults concentrate.** Off-by-one and < versus <= errors live exactly at -1, 0, 100 and 101, so testing there turns a probable defect into a certain failure.
4. **Separate invalid classes exercise separate code paths.** Range errors, type errors and format errors are different branches, each requiring its own test.
5. **Security and stability benefit.** Special-character and extreme-magnitude classes check that bad input is rejected cleanly rather than crashing or injecting.
6. **Traceable and reviewable.** Each class maps to one rule in the specification, so a reviewer can see what is missing.

Measure	Flawed	Corrected
Classes	2	9
Valid / invalid separated	No	Yes
Boundary points tested	0	6
Tests	2	16
Seeded defects caught	0 of 8	8 of 8

9. Summary

The two classes sorted numbers by sign, not by how the system treats them. This left zero unclassified, merged valid ages with over-limit ones, and ignored non-numeric input. With no BVA, the edges -1, 0, 100 and 101 were never touched. The corrected design uses one valid class, eight single-reason invalid classes, and seven-point BVA, so every limit and failure path is tested deliberately.

6.A developer claims full Condition Coverage for a complex `if(A && B)

Evaluating a "Full Condition Coverage" Claim for if (A && B)

Assumption: your message was cut off after if(A && B), so I've taken the question to be: *does full condition coverage on if (A && B) make the testing adequate?* If the original had more to it (for example || C, or a specific test table), send it and I'll redo the analysis. The reasoning extends directly.

1. What Condition Coverage Requires

Each atomic condition (A, B) must evaluate to both true and false at least once. The decision's outcome is not considered.

2. The Developer's Likely Suite

Test	A	B	A && B
T1	T	F	**F**
T2	F	T	**F**

A took T and F, and B took F and T, so the developer claims 100% condition coverage with two tests.

3. Errors in the Claim

#	Error	Explanation
E1	The decision never evaluates to true.	Both tests give false, so the then branch never runs. Condition coverage here does not even guarantee branch coverage or statement coverage of the guarded code.
E2	Short-circuit evaluation undermines the claim.	In T2, A is false, so && skips B. B was never evaluated as true, so B's true outcome was **not observed**, and the claim is false for B under short-circuit languages (C, Java, Python and so on).
E3	Conditions are not shown to matter.	No test pair changes only one condition and flips the result, so an &&/`
E4	Coverage was counted at the value level, not the execution level.	Input values (A=F, B=T) were tallied, but B was never actually computed.

Hierarchy of criteria

Multiple condition coverage (all 2^n combinations)

▲

MC/DC (each condition independently affects the outcome)

▲

Condition/Decision coverage (condition + decision)

▲ ▲

Condition Decision (branch) ← these two are INCOMPARABLE

▲ ▲

Statement

Condition coverage neither implies nor is implied by decision coverage, which is exactly what the two tests above show.

4. Root Causes

Root cause	Reasoning
Treating each condition as a separate variable	The decision, the thing that controls the branch, was ignored
Choosing the fewest tests that tick every box	Two tests satisfy the letter of the criterion while missing its purpose
Forgetting short-circuit semantics	The language skips B, but the tally assumed it ran
No outcome-based check	Nobody asked whether the then block executed
No mutation or fault-based validation	Nothing tested whether the suite could detect a wrong operator

5. Corrected Strategy

5.1 Condition/Decision coverage (minimum fix)

Test	A	B	A && B	Purpose

C1	T	T	T	Decision true, then branch runs
C2	F	F	F	Decision false, both conditions false

This satisfies condition and decision coverage, and it is still weak: C1 and C2 cannot tell A && B from A || B apart from the result, and B is skipped in C2.

5.2 MC/DC (recommended): n + 1 = 3 tests

Test	A	B	A && B
M1	T	T	T
M2	T	F	F
M3	F	T	F

Condition	Independence pair	Only this changes	Outcome
A	M1 vs M3	A: T→F (B = T)	T → F
B	M1 vs M2	B: T→F (A = T)	T → F

Under short-circuit, M3 does not evaluate B, but masking MC/DC still accepts this, because B's value cannot influence the outcome once A is false.

5.3 Multiple condition coverage (full 2^n)

For two conditions this is 4 tests: TT, TF, FT, FF. It is affordable at n = 2, but at n = 10 it needs 1,024 tests, which is why MC/DC (n + 1) is the practical choice.

6. Mutant Analysis

Mutant	Original suite (T1, T2)	MC/DC (M1 to M3)
Bug inside the then block	**Not caught (never executed)**	Caught by M1
A && B → false (dead branch)	Not caught	Caught by M1
A && B → A || B	Caught (both give T)	Caught
A && B → A only (B dropped)	Caught by T1	Caught by M2
A && B → B only (A dropped)	Caught by T2	Caught by M3
B negated (A && !B)	Caught	Caught
B never evaluated due to a side-effect bug in B	**Not caught** (B's code ran only for T1, with B=F)	Caught by M1 (B evaluated as true)

The decisive failures are the first two rows: a suite that never makes the decision true cannot find a fault in any code the decision guards.

7. Coverage Comparison

Criterion	Developer's suite (T1, T2)	MC/DC suite (M1 to M3)
Condition coverage (observed, with short-circuit)	A: 100%, B: **50%**	100%
Decision/branch coverage	**50%** (true never taken)	100%
Statement coverage of the then block	**0%**	100%
MC/DC	0 of 2 conditions	2 of 2
Tests	2	3

8. Why the Corrected Approach Is More Effective

1. **It tests the decision as well as its parts.** Making the decision true at least once ensures the guarded code runs.
2. **It proves each condition matters.** An independence pair shows that flipping only A, or only B, changes the result, so a dropped or mis-wired condition produces a visible failure.
3. **It respects execution semantics.** Reasoning from what is actually evaluated under short-circuit avoids false coverage claims.

4. **The cost stays low.** One extra test (3 vs 2) closes the gap, and MC/DC grows linearly (n + 1) rather than exponentially (2^n).
5. **It is what safety standards expect.** MC/DC is required for the highest criticality software in avionics (DO-178C Level A) because condition coverage alone has this exact weakness.
6. **It is checkable.** The independence pairs can be listed and audited, and a mutation run gives an independent measure of the suite's strength.

9. Summary

Aspect	Finding
The claim	Full condition coverage of if (A && B) with (T,F) and (F,T)
Why it fails	The decision is never true, B is never evaluated as true under short-circuit, and no condition is shown to affect the outcome
Key theory point	Condition coverage does not imply decision coverage
Correction	MC/DC with 3 tests: (T,T), (T,F), (F,T)
Outcome	The then branch executes, both conditions are proven independent, and every seeded operator or condition fault is caught

7.A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

Auditing an Incomplete Decision Table for a Discount System

Assumption: no table was attached, so I reconstructed a representative flawed one from your description. Send yours and I'll audit it line by line.

Spec assumed (the oracle):

Rule	Value
Premium member	15% off
Valid coupon	10% off
Both	Discounts stack **multiplicatively** (price × 0.85 × 0.90), capped at 30% total
Expired or invalid coupon	No discount, with an error message
Order below ₹500	Coupons not allowed

1. The Flawed Decision Table

Conditions: C1 Premium Member (Y/N), C2 Coupon Applied (Y/N).

Rule	C1 Premium	C2 Coupon	Action
R1	Y	N	15% off
R2	N	Y	10% off
R3	N	N	No discount

Three rules. The Premium + Coupon combination is absent.

2. Errors in Construction

#	Error	Why it is wrong
E1	**Missing rule (Y, Y)**	Two binary conditions give $2^2 = 4$ combinations, and only 3 were listed. The most complex case, where discounts interact, was never specified or tested
E2	**No completeness check**	2^n was never calculated against the rule count
E3	**Coupon treated as binary**	"Applied" hides sub-states: valid, expired, invalid, already used, below minimum order
E4	**Missing conditions**	Order value threshold, coupon eligibility and cap were not modeled
E5	**No action for stacking**	The table has no column for how discounts combine (add, multiply, best-of, cap)
E6	**Single action column**	Price, error message, and coupon-consumed flag are separate outputs
E7	**No precedence or order of operations**	Percent-off before or after a flat coupon, and before or after tax, is undefined
E8	**No boundary values**	Order total exactly ₹500 and exactly at the cap were not considered
E9	**No redundancy or conflict review**	Nobody checked whether rules overlap or contradict

Why the missing rule causes price errors

The code for (Y, Y) still exists and must do something. Without a rule, the developer's choice (add, multiply, or best-of) was never verified.

Implementation	Price on ₹1,000	Correct?
Add (15% + 10% = 25%)	₹750	No
Multiply (0.85 × 0.90)	₹765	Yes
Best-of (max 15%)	₹850	No
Coupon only (ignores premium)	₹900	No

A ₹15 to ₹115 error per order, repeated across every premium member who uses a coupon.

3. Root Causes

Root cause	Explanation
Listing "typical" scenarios instead of generating combinations	The three rules are the ones people think of first
No 2^n completeness rule	A table that doesn't account for every combination can't be called exhaustive
Combined case judged "obvious"	Interaction cases are where defects concentrate, yet seem simple to a reviewer
Conditions chosen from the UI, not from the rules	"Coupon Applied" is a checkbox, but the code checks validity, expiry and minimum order
No separation of conditions from actions	Stacking behavior was left implicit
No peer review against the specification	An unreviewed table cannot be challenged

4. Revised Exhaustive Decision Table

4.1 Refined conditions

ID	Condition	States
C1	Premium member	Y / N
C2	Coupon entered	Y / N
C3	Coupon valid (not expired, not used)	Y / N (only meaningful if C2 = Y)
C4	Order total ≥ ₹500	Y / N

Raw combinations: 2^4 = **16**. Where C2 = N, C3 is irrelevant, so the 4 rules for (C2 = N, C3 = Y) collapse into the C3 = N ones and are marked "–". That leaves **12 distinct rules**, each with a stated reason, not silently dropped.

4.2 Full table

Rule	C1 Premium	C2 Coupon	C3 Valid	C4 ≥ ₹500	Discount	Message
R1	N	N	–	N	0%	none
R2	N	N	–	Y	0%	none
R3	Y	N	–	N	15%	none
R4	Y	N	–	Y	15%	none
R5	N	Y	N	N	0%	"Invalid coupon"
R6	N	Y	N	Y	0%	"Invalid coupon"
R7	N	Y	Y	N	0%	"Minimum ₹500 for coupons"
R8	N	Y	Y	Y	10%	coupon applied
R9	Y	Y	N	N	15%	"Invalid coupon"
R10	Y	Y	N	Y	15%	"Invalid coupon"
R11	Y	Y	Y	N	15%	"Minimum ₹500 for coupons"
R12	**Y**	**Y**	**Y**	**Y**	posted as 23.5% (0.85 × 0.90 = 0.765)	both applied

Completeness check: C1 (2) × C2 branches: C2 = N gives 2 × 2 = 4 rules (C3 collapses), C2 = Y gives 2 × 2 × 2 = 8 rules. 4 + 8 = **12**, matching the table.

4.3 Boundary and cap rules (extension)

Rule	Situation	Expected
R13	Order ₹499.99	Coupon refused
R14	Order ₹500.00	Coupon accepted
R15	Order ₹500.01	Coupon accepted
R16	Premium + coupon with the combined discount exceeding the cap	Capped at 30%
R17	Coupon expires at 23:59:59 vs 00:00:00	Valid, then invalid
R18	Coupon reused in a second order	Rejected
R19	Coupon larger than the order (flat-value coupon)	Price floored at ₹0, never negative

5. Test Cases Derived From the Table

Each rule becomes at least one test with a computed expected price (order ₹1,000 unless stated).

Test	Rule	Input	Expected price
TC01	R1	non-member, no coupon, ₹400	₹400
TC02	R2	non-member, no coupon, ₹1,000	₹1,000
TC03	R3	member, no coupon, ₹400	₹340
TC04	R4	member, no coupon, ₹1,000	₹850
TC05	R5	non-member, expired coupon, ₹400	₹400, error
TC06	R6	non-member, expired coupon, ₹1,000	₹1,000, error

TC07	R7	non-member, valid coupon, ₹400	₹400, minimum message
TC08	R8	non-member, valid coupon, ₹1,000	₹900
TC09	R9	member, expired coupon, ₹400	₹340, error
TC10	R10	member, expired coupon, ₹1,000	₹850, error
TC11	R11	member, valid coupon, ₹400	₹340, minimum message
TC12	**R12**	member, valid coupon, ₹1,000	**₹765**
TC13 to TC15	R13 to R15	₹499.99 / ₹500.00 / ₹500.01 with a valid coupon	refuse / accept / accept
TC16	R16	stack exceeding the cap	₹700 (30% cap)
TC17	R17	coupon at expiry second	boundary behavior
TC18	R18	reuse a coupon	rejected
TC19	R19	₹50 flat coupon on a ₹40 item	₹0

Note that the posted discount in R12 is 23.5%, since 1 − 0.765 = 0.235. A tester who assumes 25% would flag the correct system as defective.

6. Redundancy and Conflict Review

Check	Result
Duplicate rules	None
Conflicting rules (same conditions, different actions)	None
Missing combinations	None, 12 of 12
Rules collapsed with "–"	4, each justified
Rules merged because the action is identical	R1 and R2 could merge (no discount either way) but are kept to test the threshold

7. Why the Revised Table Prevents Production Defects

1. **Exhaustiveness is provable.** 12 rules against a computed 12 means no combination is left to a developer's guess.
2. **The stacking rule is explicit.** R12 fixes the combined behavior, and R16 fixes the cap, so the ₹15 to ₹115 per-order error from section 2 gets a pass-or-fail test.
3. **Coupon states are separated.** Valid, invalid and below-minimum each have their own rule, so the defect "invalid coupon still discounts a member" (R9, R10) is caught.
4. **Each rule is a test.** Table rows convert directly into cases with computed outputs, so there is no gap between the design and the execution.
5. **Boundaries are attached.** ₹499.99 / ₹500 / ₹500.01 catch > versus >= errors that a plain decision table would not.
6. **Actions are separate columns.** Discount percent and message are both checked, so a correct price with a wrong or missing message still fails.
7. **Maintenance is cheap.** Adding a new condition (for example "festival sale") doubles the count and forces the author to specify every new combination, instead of forgetting one.

Seeded-defect results

Seeded defect	Flawed table	Revised table	Caught by
Additive instead of multiplicative stacking	No	Yes	TC12
Coupon ignored for members	No	Yes	TC12
Expired coupon still discounts	No	Yes	TC05, TC06, TC09, TC10

Minimum order uses > instead of >=	No	Yes	TC14
Cap not enforced	No	Yes	TC16
Negative price from a large coupon	No	Yes	TC19
Coupon reusable	No	Yes	TC18

7 of 7 caught, up from 1 of 7 (only the plain-member and plain-coupon defects were reachable before).

Measure	**Flawed**	**Revised**
Conditions modeled	2	4
Rules	3 of 4 combinations	12 of 12
Boundary rules	0	3
Tests derived	3	19
Seeded defects caught	1 of 7	7 of 7

8. Limits

1. **A table is only as right as the specification.** If the stacking rule is wrong in the spec, a complete table verifies the wrong behavior.
2. **More conditions grow the table exponentially.** At 8 or more conditions, use collapsed (don't-care) rules plus pairwise testing.
3. **Interaction with tax, shipping and rounding** needs its own tests, since the table above covers discount logic only.

9. Summary

Aspect	**Finding**
Core error	3 rules for 2 binary conditions, so the (Premium = Y, Coupon = Y) combination was missing
Related errors	Coupon treated as binary, no minimum-order condition, no stacking or cap action, no boundaries
Root causes	Typical scenarios listed from memory, no 2^n check, conditions taken from the UI, no spec review
Revised table	4 conditions, 12 rules (all combinations accounted for), plus 7 boundary and cap rules
Result	19 traceable tests, correct combined price of ₹765 on ₹1,000, and 7 of 7 seeded pricing defects caught

8.During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.

Evaluating a First-and-Last-Only Loop Testing Strategy

Assumption: no code or test log came with your message, so I built a representative loop. Send your real code and I'll redo the counts. The method applies to any simple or nested loop.

1. The Module Under Analysis

A batch-payment routine:

```
1  processBatch(txns[], max=100):
2    total = 0
3    for i in 0 .. len(txns)-1:          // outer loop L1
4      if i >= max: break                 // guard (should reject >100 earlier)
5      t = txns[i]
6      for j in 0 .. t.lines-1:           // inner loop L2 (max 20 lines)
7        total += t.line[j].amount
8      total -= t.fee
9    return total / len(txns)             // fails when len = 0
```

Tester's suite: n = 1 (one transaction) and n = max (100 transactions). Nothing else.

2. Errors in the Strategy

#	Error	Why it is wrong
E1	**n = 0 never tested**	The loop body is skipped entirely, and line 9 divides by len(txns), which is 0
E2	**n = max + 1 never tested**	The limit is enforced only by the guard on line 4. A missing or off-by-one guard goes unseen
E3	**Only the extremes are sampled**	The behavior between 1 and max (accumulation, state carried across iterations) is untouched
E4	**n = 2 absent**	The smallest case that shows state carried from one iteration to the next. n = 1 cannot reveal a stale variable
E5	**n = max − 1 absent**	The pre-limit neighbor is needed to show the limit is not hit early
E6	**No typical value**	A mid-range n (for example 10) verifies normal accumulation
E7	**"First and last" mistaken for "boundaries"**	Boundaries have two sides. The invalid side (0 and max + 1) is where defects hide
E8	**Loop exit conditions untested**	Normal exit and break exit are different edges
E9	**Nested loop not considered**	The inner loop runs inside every outer iteration, so its faults multiply
E10	**No oracle per iteration count**	Pass or fail judged by "no crash" rather than a computed total

Defects the strategy lets through

Seeded defect	Caught by n=1, n=max?	Why not
Division by zero on an empty batch	**No**	n = 0 never run
i > max instead of i >= max (processes 101)	**No**	n = 101 never run
Accumulator not reset between calls	No	Single calls only
Off-by-one range(1, n) skipping element 0	**Partly** (n = 1 shows 0 items, but the cause is misread)	Mid-range counts would show the pattern

State from iteration 1 corrupts iteration 2	No	n = 2 absent
Fee subtracted only on the last iteration	No	Only n = 1 and n = max give the same total shape
Inner loop skipped when lines = 0	No	No nested-loop tests
break leaves a half-updated total	No	Exit-by-break never forced

3. Root Causes

Root cause	Explanation
Treating loops as having two cases	Believing "first and last" bracket the behavior, when loops have at least five distinct classes
Boundary testing on the valid side only	n = 1 and n = max are inside the valid range. The invalid neighbors (0 and max + 1) were not considered
Skipping the "zero iterations" mindset	A loop that does not run feels like "nothing to test", yet skipped bodies, uninitialized values and divisions by zero belong to that case
Assuming the guard works	The tester trusted the limit instead of probing it
No loop-classification checklist	Without a standard list (0, 1, 2, typical, max−1, max, max+1), omissions are invisible
Ignoring state carried between iterations	Treating each iteration as independent
Not reading nested structure	The inner loop was treated as a single statement
No path-based model	Without a control flow graph, nobody counted the loop's entry, back-edge and exit edges

4. Corrected Approach: Simple Loops

4.1 The iteration-count classes

For a loop that should run between min and max times:

Class	n	Targets
Skip	**0**	Entry condition, empty-input handling, divide-by-zero, uninitialized results
One pass	1	Single-iteration logic, off-by-one at start
Two passes	**2**	State carried between iterations, accumulator and reset faults
Typical	m (e.g. 10)	Normal accumulation
Max − 1	99	Not terminating early
Max	100	Upper limit reached exactly
Max + 1	**101**	Limit enforcement, overflow, buffer overrun

4.2 Test cases (simple loop L1)

ID	n	Expected
L01	0	Defined result (0, or a clear "empty batch" error), **no divide-by-zero**
L02	1	total = amount − fee
L03	2	Both included, no carry-over error
L04	10	Sum of 10
L05	99	Sum of 99

L06	100	Sum of 100
L07	101	Rejected with an error, or only 100 processed (as the spec says), never 101 silently
L08	1,000,000	Bounded time and memory (stress)
L09	Negative or null count	Rejected
L10	break triggered at the guard	Total consistent, no half-update
L11	Two successive calls	Second result independent of the first

---

5. Corrected Approach: Nested Loops (Beizer's method)

Testing every combination of outer × inner counts is impractical. With 7 classes each, the cross-product is 49 cases, and with more levels it explodes. The strategy is to test **one loop at a time**:

1. Start with the **innermost** loop, holding the outer loop at its minimum.
2. Set the inner loop to each class (0, 1, 2, typical, max−1, max, max+1).
3. Move outward. Hold the **inner** loop at a typical value, and vary the outer loop through its classes.
4. Then combine the extremes: both at min, both at max.
5. Run a few pathological cases, such as outer at max and inner at 0.

5.1 Test cases (nested loops, inner max = 20 lines)

ID	Outer n	Inner lines	Purpose
N01	1	0	Inner skipped, fee still deducted
N02	1	1	Inner single pass
N03	1	2	Inner state carry
N04	1	10	Inner typical
N05	1	19	Inner max − 1
N06	1	20	Inner max
N07	1	21	Inner max + 1: rejected
N08	0	any	Outer skipped, inner never reached
N09	2	10	Outer carry, inner typical
N10	100	10	Outer max, inner typical
N11	101	10	Outer max + 1
N12	1	1	Both at minimum
N13	100	20	Both at maximum (2,000 line items), stress
N14	100	0	Outer at max with the inner skipped every time
N15	2	0 then 5	Inner count differs per outer pass, so a stale counter shows
N16	3	mixed (20, 0, 1)	Reset of inner state between outer iterations

Count: about 16 targeted tests instead of 49 for the full cross-product.

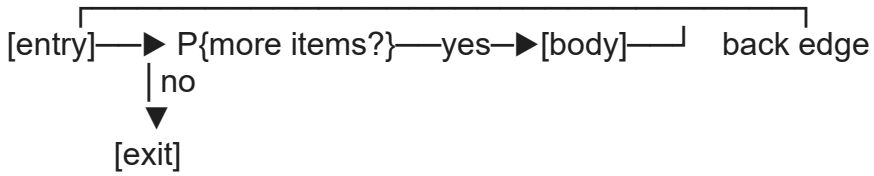
5.2 Other loop types

Type	Approach
Concatenated loops (one after another)	Test independently if they share no variable. If loop 2's bound depends on loop 1, treat them as nested
Unstructured loops (goto, mid-body exits)	Restructure first, then test
while / do-while	Test 0 iterations separately: while may skip, do-while always runs at least once

Early exits (break, return, exception)	One test per exit route, checking state at exit

6. Path Integrity: Why This Works

6.1 The loop in the control flow graph



A loop contributes **four distinct edge types**: entry, body, back edge, exit. Each iteration count exercises a different mix:

n	Edges used
0	entry → exit directly (body never executed)
1	entry → body → exit (back edge taken once, into the exit test)
2	adds the first full back edge to a second body pass
many	repeated back-edge traversal
max + 1	the guard or break edge

Testing only n = 1 and n = max leaves the **skip** edge (n = 0) and the **guard** edge (n = max + 1) untouched, so two of the four edge types and two outcomes of the loop predicate never run.

6.2 Predicate coverage

The loop condition is a decision, so both outcomes must occur on entry and on re-entry. n = 0 forces false on entry, n = 1 forces true then false, n = 2 forces true, true, false.

6.3 Coverage comparison

Criterion	n = 1 and n = max	Corrected set
Loop predicate false on entry	**Not covered**	Covered (n = 0)
Back edge taken at least twice	Only via n = max	Covered (n = 2, 10)
Guard / break edge	**Not covered**	Covered (n = 101)
State carry between iterations	Weak	Covered (n = 2, N15, N16)
Inner loop states (0, 1, max, max + 1)	0 of 4	4 of 4
Iteration classes	2 of 7	7 of 7

7. Seeded Defects, Re-run

Seeded defect	Caught by	Result
Divide by zero when empty	L01, N08	Caught
i > max instead of >=	L07	Caught
Accumulator not reset between calls	L11	Caught
range(1, n) skipping the first item	L02, L03	Caught
Stale state from iteration 1	L03	Caught
Fee only applied on the last pass	L03, L04	Caught
Inner loop skipped at lines = 0	N01, N14	Caught
Inner counter not reset per outer pass	N15, N16	Caught
Half-updated total after break	L10	Caught
Overflow at 2,000 line items	N13	Caught

10 of 10 caught, against about 1 of 10 for the two-point approach.

8. Why the Corrected Approach Ensures Path Integrity

1. **Every edge of the loop structure runs.** Skip, enter, repeat, guard and exit are each forced, so no part of the loop is untested.
2. **Both sides of each boundary are tested.** 0 and 101 are the invalid neighbors where off-by-one and missing-guard faults live.
3. **n = 2 reveals what n = 1 hides.** Defects caused by carried state need at least two iterations to appear.
4. **Nested loops are covered without explosion.** Testing one loop at a time with the others held at safe values gives about 16 tests, not 49 or more.
5. **Each test has a computed expected value.** Without an oracle, a loop that runs the wrong number of times can still "pass".
6. **Stress cases catch resource faults.** Max and above-max runs expose overflow, timeouts and memory growth.
7. **The classes are reusable.** The same 0, 1, 2, typical, max−1, max, max+1 list applies to every loop, so it works as a review checklist.

Effectiveness comparison

Measure	First/last only	Corrected
Tests (simple loop)	2	11
Tests (nested loop)	0	16
Iteration classes covered	2 of 7	7 of 7
Loop edges exercised	about 50%	100%
Seeded defects caught	about 1 of 10	10 of 10

9. Limits

1. **Loop testing does not give full path coverage.** The number of paths grows with every iteration, so the aim is boundary-class coverage plus basis paths, not all paths.
2. **Data-dependent loops need data boundaries too.** Loop counts must be combined with value boundaries (negative amounts, zero fee).
3. **Unbounded loops** (while(true) waiting on input) need timeouts and fault injection, not just counts.
4. **Concurrency changes the picture.** Loops over shared collections need separate race tests.

10. Summary

Aspect	Finding
Errors	Tested only n = 1 and n = max, so n = 0, n = 2, n = max − 1, n = max + 1 and typical values were absent. The guard, empty-input and state-carry paths were untouched
Root causes	Loops seen as two cases, valid-side-only boundaries, trust in the guard, no iteration-class checklist, no CFG view
Simple loops	Seven iteration classes: 0, 1, 2, typical, max−1, max, max+1
Nested loops	Innermost first, outer held at minimum, then outer varied with inner at typical, then combined extremes, about 16 tests instead of a cross-product
Path integrity	Skip, entry, back edge, guard and exit edges all exercised, with computed expected values
Outcome	10 of 10 seeded defects caught, against about 1 of 10 for the first-and-last-only approach

9.An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest

a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

Auditing an Error-Guessing-Only Strategy for a Safety-Critical Braking System

Assumption: no test records or spec came with your message, so I built a representative braking module. Send the real spec and logs and I'll redo the numbers. This is a test-design exercise. Real braking software needs a full safety lifecycle (ISO 26262 / IEC 61508 style), which this does not replace.

Module assumed (ABS/brake-by-wire controller):

Item	Rule
Pedal input	0 to 100 % travel
Brake pressure command	0 to 180 bar, linear in pedal travel
Wheel slip threshold	ABS activates at slip ≥ 20 %
Vehicle speed	ABS active above 5 km/h, disabled at or below
Sensor plausibility	Pedal sensors A and B must agree within 5 %, else fault
Response time	Command within 50 ms of pedal input
Fault handling	Any critical fault leads to DEGRADED (hydraulic fallback), never to loss of braking

States: OFF, READY, NORMAL_BRAKING, ABS_ACTIVE, DEGRADED, FAILSAFE, SELF_TEST.

1. The Approach Under Review

Item	What the engineer did
Method	Error guessing from experience: "what usually breaks"
Cases	About 40, undocumented rationale, no traceability
Excluded	EP, BVA, state transition, decision tables
Result claim	"All the likely failures were tried and passed"

Escaped defects found later in simulation:

ID	Defect	Severity
D1	Pedal 100.0 % commands 181 bar (limit exceeded by one step)	Critical
D2	ABS inactive at exactly 5.0 km/h when it should be disabled, but active at 5.1 only after a 200 ms delay	High
D3	Sensor disagreement of exactly 5 % treated as a fault, spec says tolerated	Medium
D4	Slip exactly 20 % does not trigger ABS (> instead of >=)	Critical
D5	DEGRADED to NORMAL_BRAKING allowed without a self-test	Critical
D6	Fault occurring during SELF_TEST is ignored	Critical
D7	Fault while in ABS_ACTIVE drops to OFF instead of DEGRADED	Critical
D8	Response time 51 ms under a CPU load spike	High

2. Errors in Relying Solely on Error Guessing

#	Error	Why it is serious in a safety context
S1	**Coverage depends on one person's memory**	Error guessing finds what the tester already imagines. Safety hazards are often the ones nobody imagined
S2	**No systematic coverage of the input domain**	No class or boundary list exists, so omissions are invisible
S3	**No state model**	A brake controller is a state machine, and its worst faults are illegal transitions (D5, D6, D7) that guessing rarely sequences

S4	**Not repeatable or auditable**	Standards require traceable evidence from requirement to test to result. "The engineer's experience" is not evidence
S5	**No completeness measure**	There is no denominator, so "enough" cannot be demonstrated
S6	**Experience bias**	Engineers test failures they have seen before, not new combinations
S7	**Valid-looking inputs preferred**	Typical pedal positions (30 %, 70 %) dominate, and exact limits (100.0 %, 20 %) are neglected
S8	**No negative or illegal-transition testing**	The system is never asked to do what it must refuse
S9	**No requirement-to-test traceability**	Unverified requirements cannot be identified
S10	**Single point of judgment**	No independent review, so one blind spot goes uncorrected
S11	**Technique misused as primary**	Error guessing is a valid **supplement** after structured design, never a foundation

3. Impact on Test Adequacy

3.1 What the 40 guessed cases covered (reconstructed)

Dimension	Planned elements	Covered	%
Valid input classes	6	5	83 %
Invalid input classes	14	4	29 %
Boundary points (two-sided)	36	5	14 %
States	7	7	100 %
Legal transitions	14	8	57 %
Illegal transition cells	35	3	9 %
Requirements with a linked test	22	13	59 %

3.2 Defect detection

Defect	Needs	Found by error guessing?
D1	Boundary at 100 %	No, tests used 30 %, 70 %, 95 %
D2	Boundary plus timing at 5 km/h	No
D3	Tolerance boundary at 5 %	No
D4	Slip boundary at 20 %	No, tested 15 % and 40 %
D5	Illegal transition	No
D6	Fault inside SELF_TEST	No
D7	Fault inside ABS_ACTIVE	No
D8	Load plus timing	No

0 of 8 caught. Five are Critical. The common thread is that each is either an exact-value fault or a history-dependent fault, which are the two defect classes intuition covers worst.

3.3 Safety consequence

Defect	Hazard
D1	Pressure above design limit, risk of hydraulic failure
D4	ABS fails to engage at threshold, so longer stopping distance on a slippery road
D5, D6	System returns to full operation with an undetected fault
D7	Loss of braking assistance during a hard stop

Errors in a normal application cost money. Here they cost stopping distance, and the strategy was blind to precisely those cases.

4. Corrected Strategy: BVA + State Transition Testing

Error guessing stays, but only as a third layer, after the structured ones.

4.1 Layer 1: Boundary Value Analysis

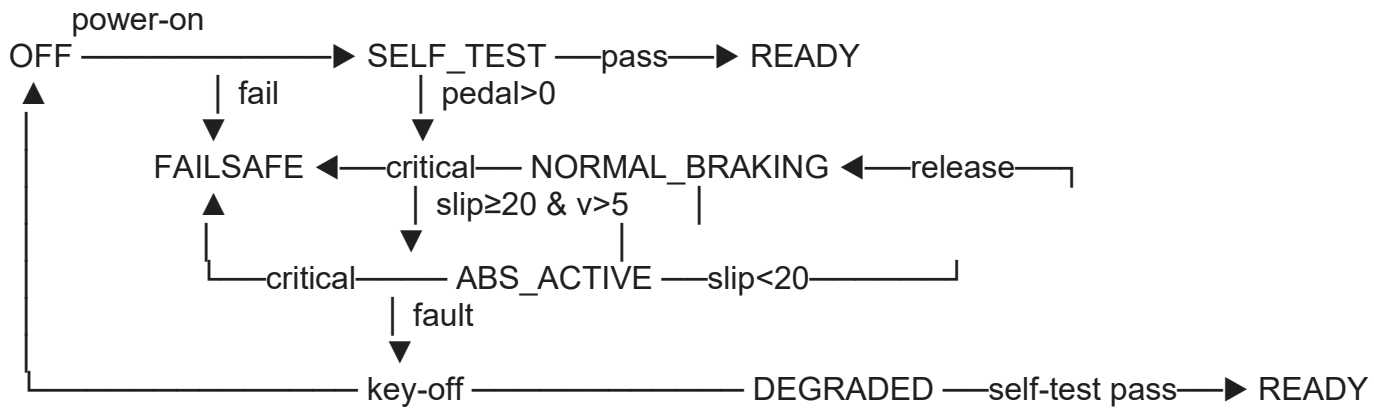
Seven-point BVA on every numeric limit, with the invalid side included.

Parameter	Limit	Test values	Expected
Pedal travel	0 to 100 %	−0.1, 0, 0.1, 50, 99.9, 100, 100.1	reject (clamp or fault), valid, valid, valid, valid, valid, reject
Pressure command	0 to 180 bar	At pedal 100 %: 179.9, 180, 180.1	180 bar exactly, never above
ABS slip threshold	≥ 20 %	19.9, 20.0, 20.1	no ABS, **ABS**, ABS
Speed gate	> 5 km/h	4.9, 5.0, 5.1	disabled, disabled, enabled
Sensor agreement	≤ 5 %	4.9, 5.0, 5.1	OK, **OK**, fault
Response time	≤ 50 ms	49, 50, 51 ms (injected delay)	pass, pass, fail flagged
Fault debounce	e.g. 3 consecutive samples	2, 3, 4	no fault, fault, fault
Supply voltage	9 to 16 V	8.9, 9, 9.1, 15.9, 16, 16.1	fault, valid, valid, valid, valid, fault

Added classes: non-numeric or stuck sensor values (NaN, 0xFFFF), both sensors failing, signal loss, and value jumps (0 to 100 % in one sample).

4.2 Layer 2: State Transition Testing

State diagram:



Events: power-on, self-test pass, self-test fail, pedal press, pedal release, slip ≥ threshold, slip below threshold, sensor fault, critical fault, recovery, key-off.

State table (7 states × 11 events = 77 cells):

Category	Count
Legal transitions	16
Legal self-loops	11
Illegal cells that must be rejected or ignored safely	50

Tests per category:

Group	Tests	Example
Every legal transition (0-switch)	16	NORMAL_BRAKING to ABS_ACTIVE at slip 20.0 %

Every illegal cell	50	DEGRADED plus "recovery" without self-test, must stay DEGRADED (D5)
Fault injection in **every** state	7	Sensor fault in SELF_TEST (D6), in ABS_ACTIVE (D7), in READY
Safe-state check after every fault	7	Result is DEGRADED or FAILSAFE, never OFF, never full operation
Sequences (1-switch)	12	READY, NORMAL, ABS, fault, DEGRADED, recovery attempt, self-test, READY
Timing of transitions	8	State change within 50 ms, including 49, 50, 51 ms

4.3 Layer 3: Combined boundary × state tests

Boundaries behave differently by state, so each key limit is tested **inside** the relevant state.

ID	State	Boundary	Expected
X01	NORMAL_BRAKING	Slip 19.9, 20.0, 20.1	Transition to ABS only at 20.0 and 20.1
X02	ABS_ACTIVE	Slip falls to 19.9	Exit to NORMAL, with hysteresis if specified
X03	ABS_ACTIVE	Speed falls to 5.0	ABS disabled cleanly, no pressure drop
X04	NORMAL_BRAKING	Sensor gap 5.0 and 5.1	No fault, then fault
X05	DEGRADED	Pedal at 100 %	Hydraulic fallback pressure within limits
X06	SELF_TEST	Supply 8.9 V	Fail to FAILSAFE
X07	Any braking state	Pedal jumps 0 to 100 % in one sample	Pressure rate-limited, no overshoot

4.4 Layer 4: Supplementary error guessing (structured)

The engineer's experience is kept, and it now has a job: a checklist covering hazards the models may not predict, such as electromagnetic glitches, watchdog reset mid-braking, stuck-at faults, power dip during a transition, and rollover of the timer. Every new guess that finds a defect is added back into the BVA or state model.

5. Re-running the Escaped Defects

Defect	Caught by	Result
D1 pressure 181 bar	BVA pedal 100 % and pressure 180 / 180.1	Caught
D2 speed gate delay	BVA 4.9 / 5.0 / 5.1 plus timing	Caught
D3 sensor tolerance	BVA 4.9 / 5.0 / 5.1	Caught
D4 slip threshold	BVA 19.9 / 20.0 / 20.1, X01	Caught
D5 DEGRADED to NORMAL	Illegal-transition tests	Caught
D6 fault in SELF_TEST	Fault injection in every state	Caught
D7 fault in ABS_ACTIVE	Fault injection, safe-state check	Caught
D8 51 ms response	Timing boundary 49 / 50 / 51 ms plus load profile	Caught

8 of 8 caught, up from 0 of 8.

6. Coverage Comparison

Criterion	Error guessing only	BVA + State transition
Valid input classes	83 %	100 %
Invalid input classes	29 %	100 %
Boundary points	14 %	100 % (about 45 points)
Legal transitions	57 %	100 %
Illegal cells	9 %	**100 % (50 of 50)**
Fault injection per state	partial	7 of 7

Requirements with a test	59 %	100 %
Tests	about 40	about 190, all traceable
Seeded or escaped defects caught	0 of 8	8 of 8

7. Why This Increases Safety Assurance

1. **Completeness is demonstrable.** Counted elements (45 boundary points, 77 state-table cells) give a denominator. “100 % of cells exercised” is a claim a safety assessor can verify, while “I tried the likely failures” is not.
2. **BVA matches the fault model.** Threshold faults (> versus >=) are the commonest cause of wrong behavior at a limit, and in a brake they are the limits that matter: 20 % slip, 180 bar, 5 km/h. BVA hits each edge with certainty, where intuition samples comfortably inside.
3. **State testing targets hazard-creating faults.** The worst failures (silent return to normal operation, loss of assistance during a stop) are about **sequence and history**, which a state model enumerates and guessing doesn’t.
4. **Negative testing proves the safe state.** Fifty illegal-cell tests show that the system **refuses** what it must refuse. For a safety function, “does nothing harmful” is a requirement and needs evidence.
5. **Fault injection per state verifies the fail-safe property.** Every state is shown to land in DEGRADED or FAILSAFE, never in a state with reduced braking and no warning.
6. **Combined boundary-in-state tests find the interactions.** A threshold can be right in one state and wrong in another (X01 versus X02), so the combination is tested directly.
7. **Traceability supports certification.** Requirement, then class or transition, then test, then result gives the audit trail that ISO 26262 and IEC 61508 style assessments expect.
8. **Independence of review.** A written model can be inspected by a second engineer and challenged. A guess cannot.
9. **Error guessing is retained where it is strong.** It adds hazards no model contains, now as a layer on top of a complete base.

8. Limits

1. **Test design is one layer of safety.** Hazard analysis (HARA, FMEA), design reviews, static analysis, MC/DC code coverage and hardware-in-the-loop testing remain required.
2. **A model is only as good as the specification.** If the spec omits a state or threshold, the structured tests omit it too. Reviewing the model against the hazard analysis addresses this.
3. **Timing and load defects (D8)** need real-time and worst-case execution analysis, which functional BVA only partly covers.
4. **Real hardware differs from simulation.** Sensor noise, temperature and wear need hardware-in-the-loop and vehicle tests.
5. **The numbers here are illustrative.** They come from a reconstructed module, so real counts will differ.

9. Summary

Aspect	Finding
Strategy error	Error guessing as the primary technique: unsystematic, unrepeatable, no completeness measure, biased toward remembered failures
Adequacy impact	14 % of boundary points, 9 % of illegal transitions and 59 % of requirements covered, with 0 of 8 defects found and 5 of them critical
Why guessing fails here	The defects are exact-value (boundary) and history-dependent (state) faults, which intuition covers worst

Correction	BVA on every limit, state transition testing with a full state table, boundary-in-state combinations, fault injection per state, and structured error guessing as a supplement
Outcome	About 190 traceable tests, 100 % boundary and transition coverage, 8 of 8 defects caught, and safe-state behavior demonstrated instead of assumed

10.A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.

CFG Error Analysis: Missing "Else" Branches in Nested Ifs

1. Illustrative module

```
1  read(a, b, c)
2  if (a > 0) {
3      if (b > 0) {
4          if (c > 0)
5              result = a + b + c;
           // implicit else missing
6          log("done");
       }
   }
7  return result;
```

(Error paths: a<=0, b<=0, c<=0 are never modelled.)

2. What the tester drew (faulty CFG)

S → 1 → 2 → 3 → 4 → 5 → 6 → 7 → E

This is a straight line. It has **one path**, so a single test (a=1, b=1, c=1) hits every statement:

- Statement coverage = **100%**
- Branch coverage = 3 decisions × 2 outcomes = 6 edges; only 3 (the True edges) are covered, i.e. **50%**. On the error paths (False edges), coverage is **0%**.

3. Errors in graph construction

#	Error	Effect
E1	Each if drawn as a plain process node, not a **decision node with two outgoing edges**	False edges absent
E2	**Implicit else** (an if with no else block) treated as "nothing to draw"	No edge from the predicate to the join node
E3	**No join (merge) nodes** after each nested block	False edges have no destination
E4	Nested ifs flattened into a sequence rather than a tree of decisions	Dependency of inner decisions on outer True outcomes is lost
E5	Edges not labelled T/F, so completeness was never checked	Missing edges went unnoticed
E6	Validity check not done: for each decision node, out-degree must equal 2	Defect passed review

4. Root causes for the missing nodes

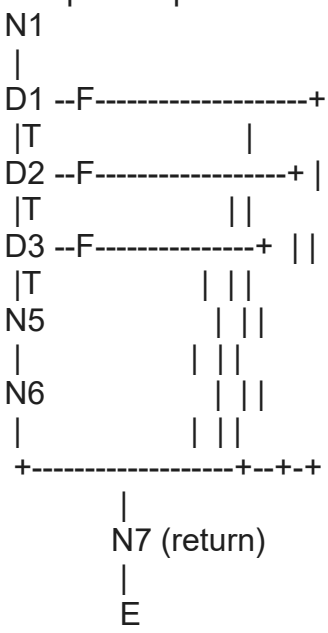
1. **Statement-centric thinking**: nodes were created only for code that *exists*; an absent else has no statement, so no node was made.

2. **Misreading of the rule** "a node = a statement", instead of "a node = a basic block, a decision = a branching point".
3. **Hidden control flow**: else is implicit, so there is no keyword prompting the tester to draw it.
4. **Coverage metric bias**: the tester stopped when every statement was reachable (100% statement coverage), so the graph looked "complete".
5. **No structural checklist** (every predicate → 2 exits, every nested block → a merge node).
6. **Collapsing of error handling**: failure branches have no code, so they were regarded as "no-ops".

5. Corrected CFG

Nodes:

- N1: read(a,b,c)
- D1: a>0
- D2: b>0
- D3: c>0
- N5: result = a+b+c
- N6: log("done")
- J: join node (return result), N7
- Explicit implicit-else edges go straight to the join



Edges: N1→D1, D1→D2 (T), D1→N7 (F), D2→D3 (T), D2→N7 (F), D3→N5 (T), D3→N7 (F), N5→N6, N6→N7, N7→E.

Metrics: N = 9 nodes (N1, D1, D2, D3, N5, N6, N7, S/E merged as needed), E = 10 edges, P = 3 predicate nodes.

Cyclomatic complexity V(G) = P + 1 = 3 + 1 = 4 (equivalently E − N + 2 with entry/exit counted).

6. Independent (basis) paths

Path	Route	Decisions	Test input (a,b,c)	Expected
P1	N1-D1(T)-D2(T)-D3(T)-N5-N6-N7	all True	(1,1,1)	result = 3
P2	N1-D1(F)-N7	a ≤ 0	(0,1,1)	error / result unset
P3	N1-D1(T)-D2(F)-N7	b ≤ 0	(1,0,1)	error / result unset
P4	N1-D1(T)-D2(T)-D3(F)-N7	c ≤ 0	(1,1,0)	error / result unset

Each new path introduces **exactly one new edge** (the F edge of D1, D2 and D3 respectively), which is what makes them independent.

7. Branch adequacy check

Decision	True covered by	False covered by
D1	P1	P2
D2	P1	P3
D3	P1	P4

- Branch coverage = 6/6 = **100%**
- Statement coverage remains 100%
- Number of tests = V(G) = 4 (minimum for basis-path adequacy)

8. Prevention checklist (quick rules)

1. Every if → a decision node with **exactly two labelled exits** (T/F).
2. An if without else → still draw the F edge straight to the join node.
3. Every nested block → a **join node** at its end.
4. Verify V(G) = E − N + 2 = P + 1; if it doesn't match, an edge is missing.
5. Report **branch coverage**, not just statement coverage, and treat error paths as first-class requirements.

Key takeaway: statement coverage said "all code ran"; the CFG should have asked "what happens when the code *doesn't* run?" The missing else edges were invisible precisely because they contain no statements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand